Chapter 2. Software Development, Testing, and Maintenance

2.0 Introduction

This chapter focuses on techniques for building larger applications, maintaining code quality, and ensuring code correctness. First up is coverage of both Apache Maven and Gradle Inc.'s eponmyous Gradle, the two main build tools that download and cache dependencies, build applications, and do much more for you in terms of automation. The remaining recipes concentrate on *testing* and *maintaining* software effectively.

2.1 Designing Applications: Packages, Modules

Problem

You want to be able to import classes and/or organize your classes, so you want to create your own package. You want to design software with a good structure that will work with the Java Module System.

Solution

Evaluate the major sections of your application, and consider its structure as one or more major modules, and packages within modules. Put a `package` statement at the front of each file, and recompile with `-d` or a build tool or IDE.

Discussion

One of the better aspects of the Java language is that it has defined a very clear packaging mechanism for categorizing and managing its large API. Modules consist of one or more packages, packages consist of classes, and classes consist of methods and fields.

As discussed in Recipe 2.2, modules provide a high degree of separation among different parts of your application. You might have three "tiers" in your application: the user interface, the middle tier or "business" tier, and the backend or data storage tier. These are good candidates for an initial set of modules. Each might initially consist of only one package, but it's good to put the module structure in place at the beginning.

Anybody can create a package, with one important restriction: you and I cannot create a package whose name begins with the four letters `java`. Packages named java. or javax. are reserved for use by Oracle's Java team and *you may not create packages starting with those names*. This is a legal obligation that you accepted when you clicked the clickwrap license agreement when you downloaded the JDK. When Java was new, there were about a dozen packages in a structure that is very much still with us, though it has quadrupled in size; some of these packages are shown in Table 2-1.

Table 2-1. Java packages basic structure

Name	Function
java.awt	Graphical user interface
java.io	Reading and writing
java.lang	Intrinsic classes (`String` , etc.)
java.lang.annotation	Library support for annotation processing
java.math	Math library
java.net	Networking (sockets)
java.nio	"New" I/O (not new anymore): channel-based I/O
java.sql	Java database connectivity
java.text	Handling and formatting/parsing dates, numbers, messages
java.time	Java 8: modern date/time API (JSR-311)
java.util	Utilities (collections, date)
java.util.regex	Regular expressions
javax.naming	JNDI
javax.print	Support for printing
javax.script	Java 6: scripting engines support
javax.swing	Modern graphical user interface

Many packages have been added over the years, but the initial structure has stood the test of time fairly well.

To create your own packages, or to add a class into an existing package, the `package` statement must be the very first noncomment statement in your Java source file—preceding even `import` statements—and it must give the full name of the package. Your package names are expected to start with your domain name backward; for example, my internet domain is *darwinsys.com*, so most of my packages begin with "com.darwinsys" and a project name. The utility classes used in this book and meant for reuse are in one of the com.darwinsys packages listed in Recipe 1.3, and each source file begins with a statement, such as this:

```
package com.darwinsys.util;
```

The demonstration classes in the *javasrc* repository do not follow this pattern. They are in packages with names related to the chapter they are in or the java.* package they relate to; for example, "lang" for basic Java stuff, "structure" for examples from the data structuring chapter (Chapter 7), "threads" for the threading chapter (Chapter 11), and so on. It is hoped that you will put them in a "real" package if you reuse them in your application!

Once you have package statements in place, be aware that the Java runtime, and even the compiler, will expect the compiled *.class* files to be found in their rightful place (i.e., in the subdirectory corresponding to the full name somewhere in your `CLASSPATH` settings). For example, the class file for `com.darwinsys.util.FileIO` must *not* be in the file *FileIO.class* in my `CLASSPATH` but must be in *com/darwinsys/util/FileIO.class* relative to one of the directories or archives in my `CLASSPATH` . Accordingly, if you are compiling with the command-line compiler, it is customary (almost mandatory) to use the `-d` command-line argument when compiling. This argument must be followed by the name of an existing directory (often . is used to signify the current directory) to specify where to build the directory tree. For example, to compile all the *.java* files in the current directory, and create the directory path under it (e.g., create *./com/darwinsys/util* in the example), use this:

```
javac -d . *.java
```

This creates the path (e.g., *com/darwinsys/util/*) relative to the current directory and puts the class files into that subdirectory. This makes life easy for subsequent compilations and also for creating archives, which is covered in Recipe 1.9.

Of course, if you use a build tool such as Maven or Gradle (see Recipe 2.4), this will be done correctly by default, so you won't have to remember to keep doing it!

Note that in all modern Java environments, classes that do not belong to a package (the anonymous package) cannot be listed in an `import` statement, although they can be referred to by other classes in that package. They also cannot become part of a JPMS module (see Recipe 2.2).

2.2 Using the Java Modules System

Problem

You are using Java 9 or later and need to deal with the Modules mechanism.

Solution

Consider adding a *module-info.java* file to relate your code to other modules.

Discussion

Java's Modules system, formerly known as Project Jigsaw, was designed to handle the need to build large applications—particularly the JDK itself—out of many small pieces. To an extent this problem had been solved by build tools like Maven (Recipe 2.4) and Gradle (Recipe 2.5), but the Modules system solves a slightly different problem than those tools. Maven or Gradle will find dependencies, download them, install them on your development and test runtime `CLASSPATH`, and package them into runnable JAR files. The Modules system is more concerned with the visbility of classes from one chunk of application code to another, typically provided by different developers who may not know or trust each other. As such, it is an admission that Java's original set of access modifiers—such as `public`, `private`, `protected`, and default visibility—was just not quite sufficient for building large-scale applications.

What follows is a brief discussion of using the Java Platform Module System (JPMS) to import modules into your application. There is an introduction to creating your own modules in Recipe 2.1. For a more detailed presentation, you should refer to a book-length treatment such as *Java 9 Modularity* by Sander Mak and Paul Bakker (O'Reilly).

Java has always been a language for large-scale development. Object orientation is one of the keys: classes and objects group methods, and access modifiers can be applied so that public and private methods are clearly separated. When developing large applications, having just a single flat namespace of classes is still not enough. Enter packages: they gather classes into logical groups within their own namespace. Access control can be applied at the package level as well so that some classes are only accessible inside a package. Modules are the next logical step up. A *module* groups some number of related packages, has a distinct name, and can restrict access to some packages while exposing other packages to different modules as part of its public API.

One thing to understand at the outset: JPMS is not a replacement for your existing build tool. Whether you provide JAR files by using Maven, Gradle, Ant, or just dump all needed JAR files into a lib directory, you still need to provide them. Also, don't confuse Maven's modules with JPMS modules; the former is the physical structuring of a project into subprojects, and the latter is something the Java platform (compiler, runtime) understands. Usually when working with Java modules, each Java module will equate to a single Maven module. There's no requirement for this; it's just a technique for retaining your sanity.

When you're dealing with a tiny, self-contained program, you don't need to be concerned with modules. Just put all the necessary JAR files on your `CLASSPATH` at compile time and runtime, and all will be well. Probably.

On some mid-life releases (Java 11 to 18), you may see warning messages like this along the way:

```
Illegal reflective access by com.foo.Bar
    (file:/Users/ian/.m2/repository/com/foo/1.3.1/foo-1.3.1.jar)
    to field java.util.Properties.defaults
Please consider reporting this to the maintainers of com.foo.Bar
Use --illegal-access=warn to enable warnings of further
 illegal reflective access operations
All illegal access operations will be denied in a future release
```

The warning message comes about as a result of JPMS doing its job, checking that no types are accessed in encapsulated packages within a module. Such messages should go away over time as all public Java libraries and all apps being developed get modularized. Meanwhile, if you see these messages, consider upgrading to a newer version of the library.

Probably?

Why will all be well only "probably"? If you are using certain classes that were deprecated over the last few releases, things won't compile. For that, you must make the requisite modules available. In the *javasrc/unsafe* subdirectory (also a Maven module), there is a class called `LoadAverage`. The load average is a feature of Unix/Linux systems that gives a rough measure of system load or busy-ness, by reporting the number of processes that are waiting to be run. There are almost always more processes running than CPU cores to run them on, so some always have to wait. Higher numbers mean a busier system with slower response.

Sun's unsupported `Unsafe` class has a method for obtaining the load average, on systems that support it. Your code has to use the Reflection API (see Chapter 17) to obtain the `Unsafe` object; if you try to instantiate `Unsafe` directly you will get a `SecurityException` (this was the case before the Modules system). Once the instance is obtained and casted to `Unsafe`, you can invoke methods such as `loadAverage()` (Example 2-1).

Example 2-1. unsafe/src/main/java/unsafe/LoadAverage.java (use of Unsafe.java)

```
public class LoadAverage {
 public static void main(String[] args) throws Exception {
   Field f = Unsafe.class.getDeclaredField("theUnsafe");
   f.setAccessible(true);
   Unsafe unsafe = (Unsafe) f.get(null);
   int numElements = 3;
   double loadAverage[] = new double[numElements];
   unsafe.getLoadAverage(loadAverage, numElements);
   for (double d : loadAverage) {
     System.out.printf("%4.2f ", d);
   }
   System.out.println();
 }
}
```

I'll show the compilation of this code shortly; but first, some configuration. If the app is using Java Modules, the *module-info.java* file has to tell the compiler and VM that the app requires use of the module with the semi-obvious name `jdk.unsupported`:

```
module javasrc.unsafe {
   requires jdk.unsupported;
      // others...
}
```

I'll say more about the module file format in Recipe 2.3.

Now that we have the code in place and the module file in the top level of the source folder, we can build the project, run the program, and compare its output against the system-level tool for displaying the load average, `uptime`. We'll still get the "internal proprietary API" warnings, but it works:

```
$ java -version
openjdk version "21.0.2" 2024-01-16
OpenJDK Runtime Environment (build 21.0.2+13-1)
OpenJDK 64-Bit Server VM (build 21.0.2+13-1, mixed mode, sharing)
$ mvn clean package
[INFO] Scanning for projects...
[INFO]
[INFO] --------------------< com.darwinsys:javasrc-unsafe >--------------------
[INFO] Building javasrc - Unsafe 1.0.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ javasrc-unsafe ---
[INFO] Deleting /Users/ian/workspace/javasrc/unsafe/target
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ javasrc-unsafe ---
[INFO] Using UTF-8 encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory /Users/ian/workspace/javasrc/unsafe/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.1:compile (default-compile) @ javasrc-unsafe ---
[INFO] Changes detected - recompiling the module!
```

```
[INFO] Compiling 2 source files to /Users/ian/workspace/javasrc/unsafe/target/classes
[WARNING] /Users/ian/workspace/javasrc/unsafe/src/main/java/unsafe/LoadAverage.java:[3,16] sun.misc.Unsafe is internal proprietary API and may be re
...
... multiple similar Java warnings and non-informative Maven messages elided ...
...
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ javasrc-unsafe ---
[INFO] Building jar: /Users/ian/workspace/javasrc/unsafe/target/javasrc-unsafe-1.0.0-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.668 s
[INFO] Finished at: 2020-01-05T14:53:55-05:00
[INFO] ------------------------------------------------------------------------
$
$ java -cp target/classes unsafe/LoadAverage
3.54 1.94 1.62
$ uptime # Just to compare against the standard user-level tool
14:54  up 1 day, 21:50, 5 users, load averages: 3.54 1.94 1.62
$
```

This gives the same numbers as the standard Unix `uptime` command when run on Java 11 through 21. As the warnings imply, it *may* (i.e., probably will) be removed "someday."

If you are building a more complex app, you will probably need to put together a more complete *module-info.java* file. But at this stage it's primarily a matter of requiring the modules you need. The standard Java API is divided into several modules, which you can list using the `java` command:

```
$ java --list-modules
java.base
java.compiler
java.datatransfer
java.desktop
java.instrument
java.logging
java.management
java.management.rmi
java.naming
java.net.http
java.prefs
java.rmi
java.scripting
java.se
java.security.jgss
java.security.sasl
java.smartcardio
java.sql
java.sql.rowset
java.transaction.xa
java.xml
java.xml.crypto
... plus a bunch of JDK modules ...
```

Of these, `java.base` is always available and doesn't need to be listed in your module file, `java.desktop` adds AWT (the older Abstract Windowing Toolkit) and Swing for graphics, and `java.se` includes basically all of what used to be public API in the Java SDK. If our load average program wanted to display the result in a Swing window, for example, it would need to add this into its module file:

```
requires java.desktop;
```

When your application is big enough to be divided into tiers or layers, you will probably want to describe these modules using JPMS, that is, create your own modules. That is the topic of Recipe 2.3.

2.3 Using JPMS to Create a Module

Problem

You want your packaged archive to work smoothly with the JPMS.

Solution

Create a *module-info.java* file in the root of the source directory.

Discussion

The file *module-info.java* was introduced in Java 9 to provide the compiler and tools with information about your library's needs and what it provides. Note that this is not a valid Java class filename because it contains a minus sign. The module also has a group of pseudokeywords, which only have their special meaning inside a module file. The simplest *module-info* is the following:

```
module foo {
   // Empty
}
```

But just as a Java class with no members won't get you very far in the real world, neither will this empty module file. We need to provide some additional information. For this example, I show how I modularized my `darwinsys-api`, a collection of 150 or so randomly accumulated classes that I reuse sometimes. Remember that Jigsaw (the module system's early name) was initially proposed as a way of modularizing the overgrown JDK itself. Most applications will need the module `java.base` (which is always included). If they need AWT, Swing, or certain other desktop-application-related classes, they also need `java.desktop`. Thus I add the following line into the module definition:

```
require java.desktop;
```

My code also has some JUnit-annotated classes and makes use of the JavaMail API, so we need those as well. JUnit, however, is only needed at test time. While Maven offers `scope`s for compile, test, and runtime, the Modules system does not.

Unfortunately, as of this writing, there are still some libraries that are not modularized. Fortunately, there is a feature known as *automatic modules*, by which if you place a JAR file on the module path that doesn't declare a module, its JAR filename will be used as the basis of an automatically generated module. At one point, it seemed there wasn't a modularized JavaMail API. At that point, I added the following:

```
require mail
```

Unfortunately, when this code was compiled, Maven's Java compiler module spit out this scary-looking warning:

```
[WARNING] ************************************************************************
[WARNING] * Required filename-based automodules detected. Please don't publish
        this project to a public artifact repository! *
[WARNING] ************************************************************************
```

However, I have since found a modularized version of each API that's needed.

The *module-info* also lists any packages that your module desires to make available, that is, its public API. So we need a series of export commands:

```
exports com.darwinsys.calendar;
exports com.darwinsys.csv;
exports com.darwinsys.database;
...
```

By default, packages that are exported cannot be examined using the Reflection API. To allow a module to introspect (use the Reflection API) on another, say, a domain model used with the Java Persistence API (JPA), use `opens`.

For example, if we have a library that needs to use Reflection on a class in `java.lang`, as indicated by the error messages when running the app, we can try to run the app again with this command-line flag:

```
java --add-opens=java.base/java.lang=ALL-UNNAMED packagename.ClassName
```

One of the purposes of Java interfaces is to allow multiple implementations of a service. This is supported in JPMS by the *service* feature. Where an API is defined as one or more interfaces in one module, and multiple implementations are provided, each in its own module, the implementation module(s) can define an implementation using `provides ... with`, as in the following:

```
exports com.darwinsys.locks;
provides com.darwinsys.locks.PessimisticLockManager
      with com.darwinsys.locks.PessimisticLockManagerImpl;
```

A module wanting to import my lock interface feature would need a `requires com.darwinsys` and might do something like this in code:

```
import java.util.ServiceLoader;
import java.util.Optional;

Optional<LockManager> opt = ServiceLoader.load(LockManager.class).findFirst();
if (!opt.isPresent()) {
   throw new RuntimeException("Could not find implementation of LockManager");
}
LockManager mgr = opt.get();
```

The use of `findFirst()` reflects the knowledge that there is only one implementation. A more complex module might offer multiple alternatives, which would require iterating on the result of the `load()` call to find the one you wanted. The `Optional` interface is described in Recipe 8.10.

The completed *module-info* for the `darwinsys-api` module is shown in Example 2-2.

Example 2-2. DarwinSys-API module-info

```
module com.darwinsys.api {

  requires transitive java.desktop;
  requires java.net.http;
  requires transitive java.prefs;
  requires transitive java.sql;
  requires java.sql.rowset;
  requires static jakarta.mail;
  requires java.xml;

  exports com.darwinsys.calendar;
  exports com.darwinsys.csv;
  exports com.darwinsys.database;
  exports com.darwinsys.diff;
  exports com.darwinsys.formatting;
  exports com.darwinsys.genericui;
  exports com.darwinsys.geo;
  exports com.darwinsys.graphics;
  exports com.darwinsys.html;
  exports com.darwinsys.io;
  opens   com.darwinsys.io;
  exports com.darwinsys.lang;
  exports com.darwinsys.locks;
  provides com.darwinsys.locks.PessimisticLockManager
    with com.darwinsys.locks.PessimisticLockManagerImpl;
  exports com.darwinsys.mail;
  exports com.darwinsys.net;
  exports com.darwinsys.notepad;
```

```
  exports com.darwinsys.numbers;
  opens   com.darwinsys.numbers;
  exports com.darwinsys.preso;
  exports com.darwinsys.reflection;
  exports com.darwinsys.regex;
  exports com.darwinsys.security;
  exports com.darwinsys.sql;
  exports com.darwinsys.swingui;
  opens   com.darwinsys.swingui;
  exports com.darwinsys.tel;
  exports com.darwinsys.testdata;
  exports com.darwinsys.tools;
  exports com.darwinsys.unix;
  exports com.darwinsys.util;
  exports com.darwinsys.xml;
}
```

See Also

JPMS is no longer new, but some library providers are still learning to use it properly. An early posting was *https://openjdk.java.net/projects/jigsaw/quick-start*. A plan for migrating to modules can be found on Jacob Jenkov's website. A discussion about preparing a multimodule Maven application is offered by Baeldung. *Java 9 Modularity* by Mak and Bakker is probably the most comprehensive treatment of JPMS.

Maven and Gradle are the two most widely used build tools for Java developers. Both help automate the build process and manage dependencies, but they have some differences that may help you decide which to use. Both are JVM-based, giving them a performance advantage over anything that has to run several separate Java processes, like a shell-script-based build system, use of the older Make tool, etc. There is detailed information on Maven in Recipe 2.4 and on Gradle in Recipe 2.5.

Maven and Gradle automatically find all the *.java files in and under *src/main/java*, compile them, find all the source files in *src/test/java*, run the tests, and create a packaged JAR/WAR/etc. file. They will stop if any step along the way fails, for example, if even a single test fails.

Both tools handle dependency management. You simply list, in the tool's config file, the name and version of each library that you need, and the build tool finds it (usually on Maven Central), downloads it over the internet, saves it in a cache folder for future use, and adds it to your `CLASSPATH` at the right time—all without your having to write any rules.

The name and version are in three parts, actually: a group ID (normally identifying the organization behind the software), the artifact ID (name of this library), and a version number (usually with three parts, like 10.0.1 for the first release of the 10th major version). These three facts together are called the *coordinates*. In Maven they are written out in XML, and in Gradle they are written on one line with colons between. In most of the book, we give the coordinates for libraries being referenced in the Gradle form, as it's more concise.

Both tools also have special knowledge of `CLASSPATH`, making it easy to set a `CLASSPATH` in various ways for compile time. Maven offers a "scope" of tests for classes and other files that will be on your `CLASSPATH` only when running tests, for example.

The Maven build tool "follows conventions over configurations," that is, it provides sensible defaults (but so does Gradle). Maven uses an XML-based configuration file named *pom.xml* to define the project's settings and dependencies. Since I still primarily use Maven, there are POM files in each of the main projects of the *javasrc* repository. Maven provides a standard project structure (which Gradle mostly adopted, for compatibility) and "lifecycle phases" such as compile, test, and package. This makes it easy for developers to get started with new projects. However, Maven build files are verbose and can be inflexible and difficult to customize.

Gradle is a newer, more flexible build tool that uses either a Groovy- or a Kotlin-based domain-specific language (DSL, also called "little language") for defining project settings and tasks. (Only the Groovy variant is used in this book, as it is the "traditional" choice for Gradle.) Gradle allows for more customization and flexibility in defining build processes, making it a good choice for complex projects. Gradle also has better support for incremental builds and parallel execution, which can improve build performance for large projects. Gradle's popularity increased when it became the only build tool supported by Android Studio for building Android applications.

In summary, Maven is a good choice for developers who prefer convention over configuration and want a simple and easy-to-use build tool. Gradle, on the other hand, is better suited for developers who need more flexibility and customization in their build processes. Ultimately, the choice between Maven and Gradle will depend on the specific requirements of the project and the preferences of the development team.

2.4 Automating Compilation, Testing, and Deployment with Apache Maven

Problem

You want a tool that does it all automatically: downloads your dependencies, compiles your code, compiles and runs your tests, packages the app, and installs or deploys it.

Solution

Use Apache Maven.

Discussion

Maven is a Java-centric build tool that includes a sophisticated, distributed dependency management system that also gives it rules for building application packages such as JAR, WAR, and EAR files and deploying them to an array of different targets. Whereas older build tools focus on the *how*, Maven files focus on the *what*, specifying what you want done.

Maven is controlled by a file called *pom.xml* (for Project Object Model). A sample *pom.xml* might look like this:

```
<project xmlns="https://maven.apache.org/POM/4.0.0"
 xmlns:xsi="https://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="https://maven.apache.org/POM/4.0.0
                     https://maven.apache.org/xsd/maven-4.0.0.xsd">
 <modelVersion>4.0.0</modelVersion>

 <groupId>com.example</groupId>
 <artifactId>my-se-project</artifactId>
 <version>1.0-SNAPSHOT</version>
 <packaging>jar</packaging>

 <name>my-se-project</name>
 <url>https://com.example/</url>

 <properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <junit5.version>5.8.1</junit5.version>
 </properties>

 <dependencies>
  <dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-api</artifactId>
      <version>${junit5.version}</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-engine</artifactId>
      <version>${junit5.version}</version>
      <scope>test</scope>
    </dependency>
   <scope>test</scope>
  </dependency>
 </dependencies>
</project>
```

This specifies a project called *my-se-project* (my standard-edition project) that will be packaged into a JAR file; it depends on the JUnit 5.x framework for unit testing (see Recipe 2.9) but only needs it for compiling and running tests. If I type `mvn install` in the directory with this POM, Maven will ensure that it has a copy of the given version of JUnit. Not just JUnit, but also anything that JUnit depends on, and anything these in turn depend on—known as *transitive dependencies*. Then it will compile everything (setting `CLASSPATH` to exclude JUnit for the main code and including it when compiling and running tests), run all the unit tests, and if they all pass, generate a JAR file for the program and install the JAR file in my personal Maven repo (under *~/.m2/repository*). This will allow other Maven projects to depend on my new project JAR file.

Note that I haven't had to tell Maven where the main and test source files live, nor how to compile them, nor where to install the JAR—this is all handled by sensible defaults, based on a well-defined project structure. The program source is expected to be found in *src/main/java*, and the tests in *src/test/java*; if it's a web application, the web root is expected to be in *src/main/webapp* by default. Of course, you can override these settings.

Note that even the preceding config file does not have to be written by hand; Maven's archetype generation rules let it build the starting version of any of several hundred types of projects. Here is how to create such a file, along with the standard directory structure and a trivial Hello, World app with a test:

```
$ mvn archetype:generate \
   -DarchetypeGroupId=org.apache.maven.archetypes \
   -DarchetypeArtifactId=maven-archetype-quickstart \
   -DgroupId=com.example -DartifactId=my-se-project

[INFO] Scanning for projects...
Downloading: https://repo1.maven.org/maven2/org/apache/maven/plugins/
   maven-deploy-plugin/2.5/maven-deploy-plugin-2.5.pom
```

```
[several dozen or hundred lines of downloading POM files and Jar files...]
[INFO] Generating project in Interactive mode
[INFO] Archetype [org.apache.maven.archetypes:maven-archetype-quickstart:1.1]
   found in catalog remote
[INFO] Using property: groupId = com.example
[INFO] Using property: artifactId = my-se-project
Define value for property version:  1.0-SNAPSHOT: : <press enter>
[INFO] Using property: package = com.example
Confirm properties configuration:
groupId: com.example
artifactId: my-se-project
version: 1.0-SNAPSHOT
package: com.example
 Y: : y
[INFO] ----------------------------------------------------------------------
[INFO] Using following parameters for creating project from Old (1.x) Archetype:
   maven-archetype-quickstart:1.1
[INFO] ----------------------------------------------------------------------
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: artifactId, Value: my-se-project
[INFO] Parameter: basedir, Value: /private/tmp
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /private/tmp/
   my-se-project
[INFO] ----------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ----------------------------------------------------------------------
[INFO] Total time: 6:38.051s
[INFO] Finished at: Sun Jan 06 19:19:18 EST 2018
[INFO] Final Memory: 7M/81M
[INFO] ----------------------------------------------------------------------
```

Alternatively, you can use `mvn archetype:generate` and select the default from a rather long list of choices. The default is a quickstart Java archetype, which makes it easy to get started.

The IDEs (see Recipe 1.4) have support for Maven. For example, if you use Eclipse, M2Eclipse (m2e) is an Eclipse plug-in that will build your Eclipse project dependencies from your POM file; this plug-in ships with current builds of Eclipse.

A POM file can redefine any of the standard goals. Common Maven goals (predefined by default to do something sensible) include the following:

`clean`
: Removes all generated artifacts

`compile`
: Compiles all source files

`test`
: Compiles and runs all unit tests

`package`
: Builds the package

`install`
: Installs *pom.xml* and the package into your local Maven repository for use by your other projects

`deploy`
: Tries to install the package (e.g., on an application server)

Most of the steps implicitly invoke the previous ones. For example, `package` will compile any missing *.class* files, and run the tests if that hasn't already been done in this run.

To run a CLI or desktop program, without having to set this up specifically in the *pom.xml* file, you can simply use the following:

```
$ mvn exec:java -Dexec.mainClass=com.example.YourClass
```

There are application server-specific targets provided by the app server vendors. As a single example, with the WildFly application server you would install some additional plug-in(s) as per their documentation, then deploy to the app server using the following:

```
mvn wildfly:deploy
```

instead of the regular deploy. Since I use this Maven incantation frequently, I have a shell alias or batch file `mwd` to automate even that.

Maven pros and cons

Maven can handle complex projects and is very configurable. I built the *darwinsys-api* and *javasrc* projects with Maven and let it handle finding dependencies, making the download of the project source code smaller (actually, moving the download overhead to the servers of the projects themselves). The downsides to Maven are that its verbose XML format makes authoring and maintaining the build files harder, and there are fewer shortcuts for common behavior. And Maven can be harder to diagnose when things go wrong. A good web search engine is your friend when things fail.

See Also

Start at the Apache Maven website.

There is an immense collection of software freely available to Maven and Gradle users just for adding a `<dependency>` element or Maven artifact into your *pom.xml* or a dependency into your *build.gradle*. You can search this repository on Maven Central or the Nexus Repository Manager.

Figure 2-1 shows a search for my *darwinsys-api* project and the information it reveals. Note that the `dependency` information listed there is all you need to have the library added to your Maven project; just copy the Dependency Information section and paste it into the `<dependencies>` of your POM, and you're done! Because Maven Central has become the definitive place to look for software, many other Java build tools piggyback on Maven Central. To accommodate these users, in turn, Maven Central offers up the dependency information in a form that half a dozen other build tools can directly use in the same copy-and-paste fashion.

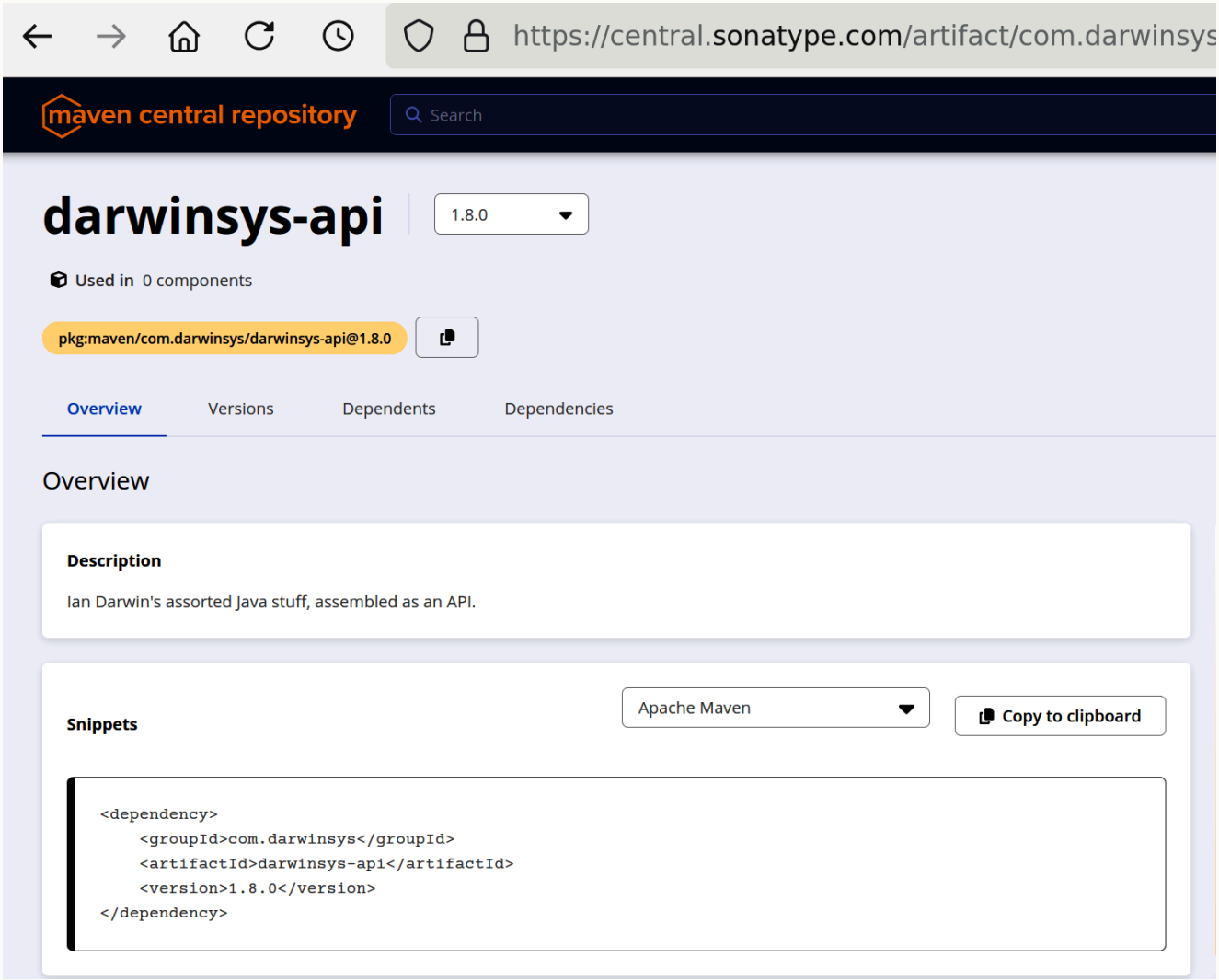


Figure 2-1. Maven Central search results

When you get to the stage of having a useful open source project that others can build upon, you may, in turn, want to share it on Maven Central. The process is longer than building for yourself but not onerous. Refer to this Maven guide or the Sonatype OSS Maven Repository Usage Guide.

One issue I fear is that a bad actor could gain access to a project's site and modify, or install a new version of, a POM. Maven automatically fetches updated POM versions. However, it does use hash signatures to verify that files have not been tampered with during the download process, and all files uploaded to Maven Central must be signed with PGP/GPG, so an attacker would have to compromise both the upload account and the signing keys.

2.5 Automating Compilation, Testing, and Deployment with Gradle

Problem

You want a build tool that doesn't make you use a lot of XML in your configuration file.

Solution

Use Gradle's simple build file format with configuration by convention for shorter build files and fast builds.

Discussion

Gradle is the latest in the succession of build tools (Make, Ant, and Maven). Gradle bills itself as "the enterprise automation tool" and has integration with the other build tools and IDEs.

Unlike Java-based tools Ant and Maven, Gradle doesn't use XML as its scripting language. Instead, it uses one of two DSLs based on the JVM-based and Java-based languages Groovy and Kotlin. To keep the examples simpler, we only cover using Groovy. You don't need to learn Groovy to use Gradle for most projects, only for those that require logic beyond the basics.

On many systems, you can install Gradle from your system's package manager. If not, you can download the latest stable version from the Gradle website, unpack the ZIP file, and add the resulting *bin* subdirectory to your path.

Then you can begin to use Gradle. If you are starting a new project from scratch, navigate into a new, empty directory and use:

```
gradle init
```

This will ask a series of questions such as what type of app to build, what to call it, what language to use (hopefully Java!) and what version thereof, what unit testing framework to use, and so on. The script will go on to create the standard directory structure common to Maven and Gradle. However, it will automatically put all that into a subdirectory called *app*, leaving just a *settings.gradle* to refer to the subdirectory with:

```
rootProject.name = 'gradledemo'
include('app')
```

The intention behind this is to allow you to build a large app out of additional modules, each of which would have its own directory and be included in the settings file.

The `init` script will also create *gradlew.bat* and *gradlew* files, the former for running Gradle under Microsoft Windows and the latter for all other operating systems. Many documents refer to running these, but if you have installed Gradle from your system's package manager, you can continue to invoke this build tool under the name `gradle`.

The *app* subdirectory will contain the main configuration file, *build.gradle*, shown in Example 2-3. The generated file will be in *app/build*. The entire generated app, along with a script of my `init` run, can be found in the *gradledemo* sudirectory of the *javasrc* repository.

For an existing project, assuming you have used the standard source directory (*src/main/java*, *src/main/test*) that is shared by Maven and Gradle, you can adapt the example *build.gradle* file in Example 2-3. You can then use it to build your app, run your unit tests, and, if tests passed and assuming it's a program rather than a library, run the application just by typing the following:

```
gradle run
```

Example 2-3. build.gradle file made by `gradle init`

```
/*
 * This file was generated by the Gradle 'init' task.
 */

plugins {
    // Apply the application plug-in to add support for building a CLI application in Java.
    id 'application'
}

repositories {
    // Use Maven Central for resolving dependencies.
    mavenCentral()
}
```

```
dependencies {
    implementation 'com.google.guava:guava:32.1.1-jre'
}

testing {
    suites {
        // Configure the built-in test suite
        test {
            // Use JUnit Jupiter test framework
            useJUnitJupiter('5.9.3')
        }
    }
}

// Apply a specific Java toolchain to ease working on different environments.
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

application {
    // Define the main class for the application.
    mainClass = 'com.example.gradledemo.App'
}
```

While the Gradle folks once tried to set up a competitor to Maven Central, the `repositories` line now lets Gradle "borrow" the industry's vast investment in Maven infrastructure.

See Also

As with Maven, there is much more functionality in Gradle. Start at Gradle's website, and see the documentation, in particular Gradle for Java and JVM projects.

2.6 Automating Dependency Management with Maven and Gradle

Problem

You need to manage your application's dependencies upon a range of third-party libraries.

Solution

Automate your dependency management with Maven or Gradle.

Discussion

As any application grows, it will need to depend on third-party libraries for some of its functionality. Those for Java are usually made available in the form of JAR files. In the early days, we would simply collect all the JAR files into one folder and write a batch script to add them all to the `CLASSPATH`. This is neither scalable nor maintainable. In today's software world, it's necessary to use tooling to manage these dependencies. Proper use of tooling will provide the following:

- Documentation on what libraries are being used
- Automatic downloading of these libraries, and verification that they are authentic
- Automatic fetching of updated versions of the libraries as fixes are made to them

Each library is described by giving its "coordinates"—the group (organization), the library name, and usually a version.

The default download site is Maven Central; running your own download site is possible but out of scope. You can find artifacts by name at Maven Central; see Figure 2-1.

Occasionally, software authors will run their own repository, in which case additional configuration will be needed—just follow their documentation.

Maven

For Apache Maven, the dependencies are listed in the `<dependencies>` section of the *pom.xml* file.

Here are some examples:

```
<project ...>
    <groupId>com.example</groupId>
    <artifactId>example_app</artifactId>
    <version>1.0.6</version>
    <packaging>jar</packaging>
    <inceptionYear>2019</inceptionYear>

    <properties>
        ...
        <darwinsys-api-version>1.9.0-SNAPSHOT</darwinsys-api-version>
        <pdfbox-version>2.0.24</pdfbox-version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.apache.pdfbox</groupId>
            <artifactId>pdfbox</artifactId>
            <version>${pdfbox-version}</version>
        </dependency>

        <dependency>
            <groupId>com.darwinsys</groupId>
            <artifactId>darwinsys-api</artifactId>
            <version>${darwinsys-api-version}</version>
        </dependency>
        ...
    </dependencies>
</project>
```

It's quite common for most libraries to have three levels of version number; the leftmost being for major revisions and the rightmost for the smallest change. These versions must be listed in `<version>` tags in the configuration file. It is somewhat common to list the version numbers of libraries in the `<properties>` section and refer to these properties in the `<version>` field, as done here. Where there are several modules from the same project, this avoids repetition and reduces the chance of missing one when updating manually.

When we build the project the first time, the two libraries listed, along with all of their dependencies, will be downloaded. Maven optionally allows a dependency to have a "scope" (where it is visible). Scope values include compile, test, and provided. The first is only on `CLASSPATH` during the build/compile phase. The second is only during testing, and the third is also compile-only, but makes clear that the artifact will be provided at runtime by a container such as a Jakarta EE server or Spring Boot.

To see what versions of these dependencies and their subdependencies are in use, you can run the following:

```
mvn dependency:tree
```

This will print all the dependencies, in a format like this:

```
[INFO] org.example:demo_application:jar:1.0.6
[INFO] +- org.apache.pdfbox:pdfbox:jar:2.0.24:compile
[INFO] |  +- org.apache.pdfbox:fontbox:jar:2.0.24:compile
[INFO] |  \- commons-logging:commons-logging:jar:1.2:compile
```

While we only listed `pdfbox`, `pdfbox` depends on `fontbox`, so that is also downloaded. `pdfbox` also requires `commons-logging`.

After working on the software for some time, we want to update to new versions of the libraries as they become available. This is important in today's internet with many bad actors attempting to exploit unpatched vulnerabilities! If the version numbers are in

`<properties>`, use:

```
mvn versions:update-properties
```

If instead the version numbers have been listed explicitly in the `<version>` elements, use:

```
mvn versions:use-latest-versions
```

There are other options for both the `dependency` and `versions` plug-ins; each will accept `help` as its argument, to list all the possibilities.

Gradle

You can easily add dependencies to a Gradle project. Many IDEs have support for doing so. Or you can just open the *build.gradle* file and add lines with the "coordinates" into the `dependencies` section (adding that section if it's missing). For example, here we've added the darwinsys-api library:

```
dependencies {
    implementation 'com.google.guava:guava:32.1.1-jre'
    implementation 'com.darwinsys:darwinsys-api:1.9.0-SNAPSHOT'
}
```

The next time you build the project, the JAR file for that library will be downloaded and cached. Unlike Maven, Gradle does not by default show such downloads:

```
$ vi app/build.gradle
$ gradle run
> Task :app:compileJava
> Task :app:processResources NO-SOURCE
> Task :app:classes

> Task :app:run
Hello World!

BUILD SUCCESSFUL in 2s
2 actionable tasks: 2 executed
$
```

You can view the dependency tree with `gradle dependencies`, but you either have to `cd` down into the *app* subdirectory, or use `gradle :app:dependencies`. This is because the *build.gradle* file is in the *app* subdirectory, and, unlike the `run` task, the `dependencies` task doesn't follow the setting that tells it to look in the *app* module. The Gradle version of this output is substantially longer than the Maven version; here's part of it:

```
$ gradle :app:dependencies

> Task :app:dependencies

------------------------------------------------------------
Project ':app'
------------------------------------------------------------

annotationProcessor - Annotation processors and their dependencies for source set 'main'.
No dependencies

compileClasspath - Compile classpath for source set 'main'.
+--- com.google.guava:guava:32.1.1-jre
|    +--- com.google.guava:guava-parent:32.1.1-jre
|    |    +--- com.google.code.findbugs:jsr305:3.0.2 (c)
|    |    +--- org.checkerframework:checker-qual:3.33.0 (c)
|    |    +--- com.google.errorprone:error_prone_annotations:2.18.0 (c)
|    |    +--- com.google.j2objc:j2objc-annotations:2.8 (c)
```

```
|   |   \--- junit:junit:4.13.2 (c)
|   +--- com.google.guava:failureaccess:1.0.1
|   +--- com.google.code.findbugs:jsr305 -> 3.0.2
|   +--- org.checkerframework:checker-qual -> 3.33.0
|   +--- com.google.errorprone:error_prone_annotations -> 2.18.0
|   \--- com.google.j2objc:j2objc-annotations -> 2.8
\--- com.darwinsys:darwinsys-api:1.8.0
    +--- org.junit.jupiter:junit-jupiter-engine:5.7.1
    |   +--- org.junit:junit-bom:5.7.1
    |   |   +--- org.junit.jupiter:junit-jupiter-api:5.7.1 (c)
    |   |   +--- org.junit.jupiter:junit-jupiter-engine:5.7.1 (c)
    |   |   +--- org.junit.platform:junit-platform-engine:1.7.1 (c)
    |   |   +--- org.junit.vintage:junit-vintage-engine:5.7.1 (c)
    |   |   \--- org.junit.platform:junit-platform-commons:1.7.1 (c)
...
... many lines elided
...
(c) - A dependency constraint, not a dependency. The dependency affected by the constraint occurs elsewhere in the tree.
(*) - Indicates repeated occurrences of a transitive dependency subtree...
(n) - A dependency or dependency configuration that cannot be resolved.

A web-based, searchable dependency report is available by adding the --scan option.

BUILD SUCCESSFUL in 556ms
1 actionable task: 1 executed
$
```

The `n` indicator can be helpful in debugging dependency failure issues.

Gradle does not yet appear to provide a dependency update plug-in. There is an open source plug-in from Ben Manes on GitHub that will report on dependencies needing updates. The easiest way to run it is to add this line into the `plugins` section of *build.gradle*:

```
id "com.github.ben-manes.versions" version "0.51.0"
```

Of course a later version may be available by the time you read this (if so, the plug-in will tell on itself!). After adding this line, you can run a report; for example:

```
$ gradle gradle :app:dependencyUpdates
------------------------------------------------------------
:app Project Dependency Updates (report to plain text file)
------------------------------------------------------------

The following dependencies are using the latest milestone version:
 - com.github.ben-manes.versions:com.github.ben-manes.versions.gradle.plugin:0.51.0

The following dependencies have later milestone versions:
 - com.darwinsys:darwinsys-api [1.5.0 -> 1.8.0]
     http://darwinsys.com/darwinsys-api/
 - com.google.guava:guava [32.1.1-jre -> 33.3.1-jre]
     https://github.com/google/guava
...
```

Unlike Maven, this will *not* actually update your *build.gradle* file; you have to do that manually (though some IDEs make it slightly easier).

2.7 Dealing with Deprecation Warnings

Problem

Your code used to compile cleanly, but now it gives deprecation warnings.

Solution

You must have blinked. Either live—dangerously—with the warnings or revise your code to eliminate them.

Discussion

Each new release of Java includes a lot of powerful new functionality, but at a price: during the evolution of these new features, Java's maintainers find some old components that were not designed well or not implemented effectively and should no longer be used because they cannot be properly fixed. In the first major revision, for example, they realized that the `java.util.Date` class had some serious limitations with regard to internationalization. Accordingly, many of the `Date` class methods and constructors are marked "deprecated." According to the *American Heritage Dictionary*, to deprecate something means to "express disapproval of; deplore." Java's developers are therefore disapproving of the old way of doing things. Try compiling this code:

```
import java.util.Date;

/** Demonstrate deprecation warning */
public class Deprec {

 public static void main(String[] av) {

  // Create a Date object for June 6, 2014
  // @SuppressWarnings("deprecation") // Uncomment to suppress
  // EXPECT DEPRECATION WARNING without @SuppressWarnings
  Date d = new Date(2014-1900, 6-1, 10);
  // Shows why java.time.LocalDate.of(2014,06,10) is safer!
  System.out.println("Date is " + d);
 }
}
```

What happened? When I compiled it (prior to adding the `@SuppressWarnings()` annotation), I got this warning:

```
C:\> javac Deprec.java
Note: Deprec.java uses or overrides a deprecated API.  Recompile with
"-deprecation" for details.
1 warning
C:\>
```

So, we follow orders. For details, recompile with `-deprecation` to see the additional details:

```
C:\> javac -deprecation Deprec.java
Deprec.java:10: warning: constructor Date(int,int,int) in class java.util.Date
has been deprecated
      Date d = new Date(2014-1900, 06-1, 10);    // June 10, 2014
                   ^
1 warning

C:\>
```

The warning is simple: the `Date` constructor that takes three integer arguments has been deprecated. How do you fix it? As in most questions of usage, the answer is to refer to the Javadoc documentation for the class. Of course, the older `Date` class has been replaced by `LocalDate` and `LocalDateTime` (see Chapter 6), so you'd only see that particular example in legacy code, but the principles of dealing with deprecation warnings matter, because some new releases of Java add deprecation warnings to parts of the API that were previously OK to use.

As a general rule, when something has been deprecated, you should not use it in any new code; and, when maintaining code, strive to eliminate the deprecation warnings.

In addition to `Date`, the main areas of deprecation warnings in the standard API are the really ancient event handling and some methods (a few of them important) in the `Thread` class.

You can also deprecate your own code when you come up with a better way of doing things. Put an `@Deprecated` annotation immediately before the class or method you wish to deprecate and/or use an `@deprecated` tag in a Javadoc comment (see Recipe 1.7). The Javadoc comment allows you to explain the deprecation.

See Also

Numerous other tools perform extra checking on your Java code. See my Checking Java Programs website.

2.8 Batch Refactoring for Warnings and Migrations

Problem

Your project generates a lot of compiler/IDE warning messages. Or, you want to migrate to a new version of Java or make other sweeping changes across a codebase.

Solution

Fix the warnings now, using the warnings as guidelines, before they get out of hand. For all of the problems just mentioned, consider using a batch refactoring tool.

Discussion

As a rule of thumb, I used to predict that, once a project achieves the dubious distinction of having 38 (or perhaps 42) warning messages from Eclipse, that project would never be clean again. (In IntelliJ, Code→Inspect Code...→Analyze on a large project will usually generate hundreds or thousands of warnings, which can be discouraging.) However, "A New Hope" has arisen. While modern IDEs tend to have good refactoring menus, these require interaction on each refactoring, which takes time. There are now some automated refactoring tools, which apply refactorings without the developer needing to interact with each warning-generating line of code. Of course, the developer needs to compare the modifications with the original (e.g., `git diff`), and run all the unit tests, before committing the change to the source repository. But such tools can be a big win in terms of time. And they can help both with maintaining clean code and in making the all-through-the-code changes in migrating from use of one API to another (e.g., TestNG to JUnit).

One such tool is OpenRewrite. OpenRewrite is unsurprisingly open source, free to use forever. And it isn't limited to Java; it supports several other languages as well, though those don't concern us.

While the software could in theory be run manually, it's much easier to run using either Maven (Recipe 2.4) or Gradle (Recipe 2.5). I'm a Maven user, so I ran OpenRewrite via Maven. You can embed OpenRewrite into your *pom.xml* or *build.gradle* file, should you intend to run certain refactorings continually. For one-time use, you can just tell Maven from the command line to run the plug-in. I ran one composite recipe—`UpgradeToJava21`—on the *javasrc* source archive. Here's how that went:

```
$ mvn -U org.openrewrite.maven:rewrite-maven-plugin:run \
      -Drewrite.recipeArtifactCoordinates=org.openrewrite.recipe:rewrite-migrate-java:RELEASE\
      -Drewrite.activeRecipes=org.openrewrite.java.migrate.UpgradeToJava21

... lots of verbiage omitted ...

[INFO] Running recipe(s)...
[WARNING] Changes have been made to main/pom.xml by:
[WARNING]    org.openrewrite.java.migrate.UpgradeToJava21
[WARNING]       org.openrewrite.java.migrate.UpgradeToJava17
[WARNING]          org.openrewrite.java.migrate.Java8toJava11
[WARNING]             org.openrewrite.java.migrate.javax.AddInjectDependencies
[WARNING]                org.openrewrite.java.dependencies.AddDependency: {groupId=jakarta.inject, artifactId=jakarta.inject-api, version=1.0.3, onlyIfUs
[WARNING] Changes have been made to main/src/main/java/ai/ChatGptApiDemo.java by:
[WARNING]    org.openrewrite.java.migrate.UpgradeToJava21
[WARNING]       org.openrewrite.java.migrate.UpgradeToJava17
[WARNING]          org.openrewrite.java.migrate.lang.StringFormatted
[WARNING]       org.openrewrite.java.migrate.util.SequencedCollection
[WARNING]          org.openrewrite.java.migrate.util.ListFirstAndLast
[WARNING] Changes have been made to main/src/main/java/database/NeverConcatenate.java by:
[WARNING]    org.openrewrite.java.migrate.UpgradeToJava21
[WARNING]       org.openrewrite.java.migrate.UpgradeToJava17
```

```
[WARNING]           org.openrewrite.java.migrate.lang.StringFormatted

... lots more like that ...

[WARNING] Please review and commit the results.
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  01:47 min
[INFO] Finished at: 2024-04-14T10:41:19-04:00
[INFO] ------------------------------------------------------------------------
$
```

In looking at the results, there are about 70 changed files. I tried compiling everything first and found that, while fixing at least 70 issues, it had introduced one, in fixing deprecation warnings! Here are the details:

```
 public static String converse(
   String host, int port, String path, String postBody) throws IOException {
-    URL url = new URL("http", host, port, path);
+    URL url = new URI("http", null, host, port, path, null, null).toURL();
     return converse(url, postBody);
 }
```

While replacing the deprecated `URL` constructor with the `URI` constructor, OpenRewrite failed to add a `throws` for `URISyntaxException`, which, unlike `MalformedURLException`, is not a subtype of `IOException`. That was the fix that launched a thousand source file compiles. Perhaps out of caution, the refactoring added `jakarta.inject` to the POM file, which isn't needed for anything. Three additional files still give deprecation warnings, but that's a small enough number to fix by hand!

I then went through `git diff` to make sure there weren't any unwanted changes, ran tests, and eventually committed most of it.

I did choose the "update" recipe for this demo because it doesn't require any further configuration. Obviously some of the recipes do need configuration, which you provide in a file named *recipe*. The OpenRewrite website hosts a full list of Java-specific recipes along with detailed documentation on each. I recommend this tool for those working on projects of any size.

OpenRewrite is funded by Moderne, a company and a platform to run OpenRewrite recipes at scale, across multiple codebases in multiple repositories. Quoting its website, "It's a place where platform teams and developers can collaborate to drive migrations across their codebases, mass-commit code fixes, and perform large-scale impact analyses." If you need to improve code at scale—across hundreds or thousands of repositories—check it out!

2.9 Maintaining Code Correctness with Unit Testing: JUnit

Problem

You want to have tooling to track down errors in your code.

Solution

Use unit testing to validate each class as you develop it.

Discussion

Stopping to use a debugger is time-consuming, and finding a bug in released code is much worse! It's better to *test* beforehand. The methodology of unit testing has been around for a long time; it is a tried-and-true means of getting your code tested in small blocks. Typically, in an object-oriented (OO) language like Java, unit testing is applied to individual classes, in contrast to system or integration testing where a complete slice or even the entire application is tested.

I have long been an advocate of this basic testing methodology. Indeed, developers of the software methodology known as Extreme Programming (XP for short) advocate *Test-Driven Development* (TDD): writing the unit tests *before* you write the code. They also advocate running your tests almost every time you build your application. And they ask one good question: *If you don't have a test, how do you know your code (still) works?* This group of unit-testing advocates has some well-known thought leaders, including Erich Gamma of

Design Patterns fame and Kent Beck of *eXtreme Programming* fame (both Addison-Wesley). I definitely go along with their advocacy of unit testing.

Indeed, many of my Java source files used to come with a "built-in" test. Classes that are not main programs in their own right would often include a `main()` method that just tests out or at least exercises the functionality of the class. What surprised me is that, before encountering XP, I used to think I did this often, but an actual inspection of two projects indicated that only about a third of my classes had test cases, either internally or externally. Plus, there was no good way to find and run these. What is needed is a uniform methodology. That is provided by JUnit.

JUnit is a Java-centric methodology for providing test cases. It is hosted at *junit.org* and normally downloaded using Maven or Gradle. It is a very simple but useful testing tool. It is easy to use—you just write a test class that has a series of methods and annotate them with `@Test`. JUnit uses introspection (see Chapter 17) to find all these methods and then runs them for you. Extensions to JUnit handle tasks as diverse as load testing and testing enterprise components; the JUnit website provides links to these extensions. All modern IDEs provide built-in support for generating and running JUnit tests.

How do you get started using JUnit? All that's necessary is to write a test. Here I have written a simple test of my `Person` class and placed it into a class called `PersonTest` (note the obvious naming pattern):

```
public class PersonTest {

 Person p;

 @Test
 public void testNameConcat() {
  String act = p.getFullName();
  assertEquals(act, "Ian Darwin", "Name concatenation");
 }

 @BeforeEach
 void setup() {
  p = new Person("Ian", "Darwin");
 }
}
```

JUnit 4 has been around for ages and works well, but is being replaced by its younger sibling. JUnit 5, also known as JUnit Jupiter (after the fifth planet), is a decade or so old and has some improvements over JUnit 4. A simple test like this `PersonTest` class will be the same in JUnit 4 or 5 (but with different imports and a different before-type annotation; the one shown is from JUnit 5). Using additional features, like setup methods to be run before each test, requires different annotations between JUnit 4 and 5. OpenRewrite (see Recipe 2.8) provides recipes for migrating from JUnit 4 to 5.

You could run `PersonTest` manually, compiling the test and invoking the command-line test harness. First, download the file *junit-platform-console-standalone-1.10.3.jar* (or a later version) from the JUnit User Guide, where you can also find directions on running it. In practice, most developers find that running tests this way is incredibly tedious, due to having to set several JARs in `CLASSPATH`. Instead, just put the tests in the standard directory structure (i.e., *src/test/java/*) with the same package as the code being tested. Then run Maven (see Recipe 2.4), which will automatically compile and run all the unit tests and will halt the build if any test fails, *every time you try to build, package, or deploy your application*. Gradle will do so too.

All modern IDEs provide built-in support for running JUnit tests; in Eclipse, you can right-click a project in the Package Explorer and select Run As→Unit Test to have it find and run all the JUnit tests in the entire project. The `MoreUnit` plug-in (free in the Eclipse Marketplace) aims to simplify the creation and running of tests. IntelliJ also finds and runs tests; the `PersonTest` is shown in Figure 2-2. The checkmarks in the lower left (green in the PDF, gray in the printed book) show that all tests passed.

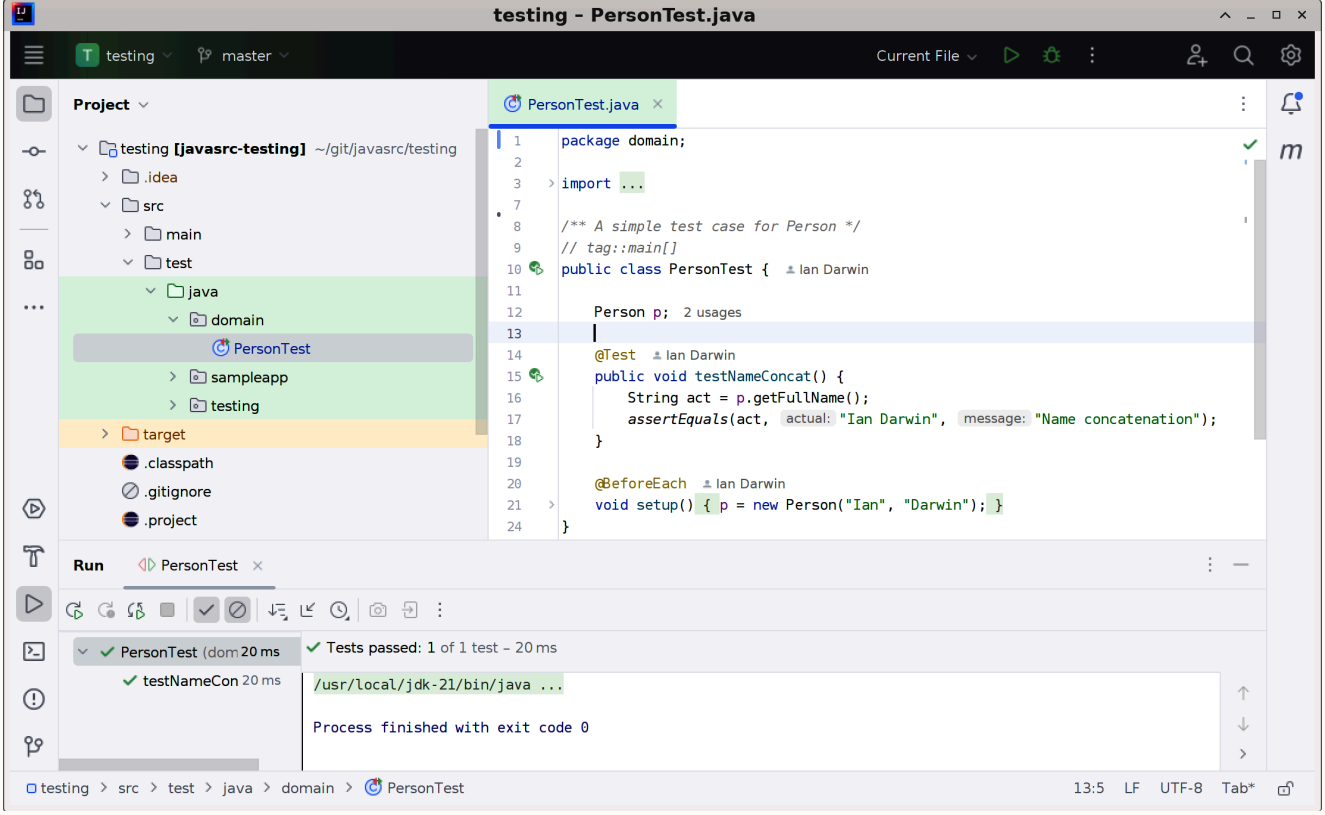


Figure 2-2. `PersonTest` run in IntelliJ

The *Hamcrest matchers* allow you to write more expressive tests at the cost of an additional download. Support for them is built into JUnit 5 via the `assertThat` static method; you just have to add them to your build file.

Here's an example of using the Hamcrest matchers:

```
/// This is really a Demo, not a Test...
public class HamcrestTest {

  @Test
  public void showGroupAssertions() {
    assertAll("Assert tests in a group",
    //  () -> assertEquals(2,3), // Sure to fail first: see if others run.
    //  () -> assertEquals(3,1+1), // Will also run if uncommented, and fail
      () -> assertEquals(2,1+1),
      () -> assertEquals("Hi", "Highlighter".substring(0,2))
    );
  }
}
```

See Also

JUnit offers considerable documentation of its own; download it from the website listed earlier.

The very ancient JUnit 3.x series predates Java 5 annotations; it used a naming convention to find and run tests. If you find any such tests, you should probably update them. OpenRewrite (Recipe 2.8) has a series of recipes for improving tests, including converting JUnit 3.x and JUnit 4 to JUnit 5. I ran this on the 65 or so tests in the *javasrc* repository; it changed all but a few of them, and left four or five (mostly JUnit 3.8 tests) needing a bit of editing before they would compile and run. Still, it saved a lot of work! In case you need it, here's the incantation I used:

```
$ mvn -U org.openrewrite.maven:rewrite-maven-plugin:run \
-Drewrite.recipeArtifactCoordinates=org.openrewrite.recipe:rewrite-testing-frameworks:RELEASE \
-Drewrite.activeRecipes=org.openrewrite.java.testing.junit5.JUnit5BestPractices
```

An alternative unit test framework for Java is *TestNG*. It got some early traction by adopting features such as Java annotations before JUnit did; but since JUnit got with the annotations program, JUnit remained the dominant package for Java unit testing.

Another package of interest is AssertJ, which appears to offer similar power to the combination of JUnit with Hamcrest.

Don't confuse any of these with the `assert` keyword, which is placed in production code rather than in test code, and is only active when the `-ea` (enable assertions) command-line option is present at runtime.

Finally, one often needs to create substitute objects for use by the class being tested (the dependencies of the class under test). While you can code these by hand, in general I encourage use of packages such as Mockito which can generate *mock objects* dynamically, have these mocks provide fixed return values, verify that the dependencies were called correctly, and so on. See Recipe 2.10 for details on Mockito.

Remember: *test early and often!*

2.10 Isolating the Test Target with Mock Objects and Mockito

Problem

When unit testing, it's desirable to avoid depending on the behavior of dependent objects used by the object you're testing.

Solution

Mock objects allow you to provide a fully controlled, fake version of a dependent object for your test subject to use, either before the dependency is written or to prevent the correctness of your test from depending on a class other than the one being tested.

Discussion

It often happens that you need to test an object that has dependencies, either before the dependency class is written or so the correctness of your test is independent of code outside the code you're testing. That's where mock objects come in. A mock object pretends to be the dependency but is really a replacement for it, concocted just for the purpose of testing.

The simplest form is the hardcoded mock; you simply write a class that implements the interface of (or possibly extends) the dependency:

```
public class MockWeatherStation implements WeatherStation {
    // implement required methods...
}
```

This is good enough to satisfy simple compilation, until you start to call methods. Then you can add the first few methods. Or have your IDE implement all the missing methods from the interface, but they will all return the default values of 0, `false`, or `null`. A more flexible method is to create a "dummy" object via the Mockito framework. This will conveniently create a mock object that either implements or extends the interface or class you provide it with. The following snippets are from *testing/src/test/java/sampleapp/WeatherInfoSupplierTest.java*:

```
import static org.mockito.*;
...
WeatherStation mockStation = mock(WeatherStation);
```

The methods will all still return the default values. But now you can have them return particular values, using Mockito's `when` function. You can also use a Mockito mock as a "mock with expectations," that is, expecting that a given method on the mock was called a certain number of times (or was never called), or that it was called with certain arguments, etc.:

```
    when(mockStation.getTemperature()).thenReturn(19.0);
    if (testCount == 0) { // hard-coded mock, no expectations
        return;
    }
    assertEquals(19.0, reporter.getTemperature());
    // Above call is expected to delegate to station.getTemperature()
    verify(mockStation, atLeastOnce()).getTemperature();
```

The `when` call specifies that when `mockStation.getTemperature()` is called (it is called by the reporter's `getTemperature()`, by simple delegation), it will return the value 19 (degrees C). This is much easier to expect in a test than whatever today's temperature is. The issue is that you can only get the current exact temperature from the actual sensor you're testing, so that would not be much of a test. Then we call the actual work method (in this case, `reporter.getTemperature()`) on the class we're testing (remember that the mock is just a bit player in this drama). Finally, we tell the mock to verify that our expectations were met regarding how the dependency was used; if not, the test will fail.

There is of course far more to Mockito. You can create mocks just by annotating fields. You can create expectations that a certain value of a certain type was passed into a method. For this and more, see the Mockito documentation.

There are other mocking frameworks, which can be found on the web.

2.11 Logging: Network or Local

Problem

Your class is running in such a way that viewing its debugging output is difficult.

Solution

Use a network-aware logger like the Java Logging API (JUL), the Apache Logging Services Project's Log4j, the open source Simple Logging Facade for Java (SLF4J), or another logger discussed here.

Discussion

Getting the debug output from a desktop client is fairly easy on most operating systems. But if the program you want to debug is running in a container like a servlet engine or an EJB server, it can be difficult to obtain debugging output, particularly if the container is running on a remote computer. It would be convenient if you could have your program send messages back to a program on your desktop machine for immediate display. Needless to say, it's not that hard to do this with Java's socket mechanism.

Many logging APIs can handle this:

- For years, Java has had a standard logging API, Java Util Logging (JUL, discussed in Recipe 2.14), that talks to various logging mechanisms, including Unix `syslog`.
- The Apache Logging Services Project produces Log4j, which is used in many open source projects that require logging (see Recipe 2.13).
- The Apache Commons Logging is a facade that lets you choose an implementation (such as JUL or Log4j). It is not discussed here.
- SLF4J (see Recipe 2.12) is the newest logging API and, as the name implies, is a facade that can use the others.
- Before these became widely used, I wrote a small, simple API to handle this type of logging function. My `netlog` is not discussed here because it is preferable to use one of the standard logging mechanisms; its code is in the *logging* subdirectory of the *javasrc* repo if you want to exhume it.

The JDK logging API, Log4j, and SFL4J are more fully fleshed out and can write to such destinations as a file; an `OutputStream` or `Writer`; or a remote Log4j, Unix `syslog`, or Windows Event Log server.

The program being debugged is the client from the logging API's point of view—even though it may be running in a server-side container such as a web server or application server—because the network client is the program that initiates the connection. The program that runs on your desktop machine is the "server" program for sockets because it waits for a connection to come along.

If you want to run any network-based logger reachable from any public network, you need to be more aware of security issues. One common form of attack is a simple denial-of-service (DoS), during which the attacker makes a lot of connections to your server in order to slow it down. If you are writing the log to disk, for example, the attacker could fill up your disk by sending lots of garbage. In common use, your log listener would be behind a firewall and not reachable from outside; but if this is not the case, beware of the DoS attack.

TIP

Some of the next few recipes include options for networking. If you're short on the details of how networking is done in Java, you might want to refer to Chapter 14. And, if what you want is to log the network traffic itself (SSL only), you can use `System.setProperty("javax.net.debug", "all")` or set this on the Java command line with `-Djava.net.debug=all`. Beware that the output will be hundreds of lines long just for opening a connection! The output can be decoded using these instructions from Oracle.

2.12 Setting Up SLF4J

Problem

You want to use a logging API that lets you use any of the other logging APIs, for example, so that your code can be used in other projects without requiring them to switch logging APIs.

Solution

Use SLF4J: get a `Logger` from the `LoggerFactory`, and use its various methods for logging.

Discussion

Using SLF4J requires only one JAR file to compile, *slf4j-api-1.x.y.jar* (where *x* and *y* will change over time). To actually get logging output, you need to add one of several implementation JARs to your runtime `CLASSPATH`, the simplest of which is *slf4j-simple-1.x.y.jar* (where *x* and *y* should match between the two files).

Once you've added those JAR files to your build script or on your `CLASSPATH`, you can get a `Logger` by calling `LoggerFactory.getLogger()`, passing either the string name of a class or package or just the current `Class` reference. Then call the logger's logging methods. A simple example is in Example 2-4.

Example 2-4. main/src/main/java/logging/Slf4jDemo.java

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class Slf4jDemo {

  final static Logger theLogger =
    LoggerFactory.getLogger(Slf4jDemo.class);

  public static void main(String[] args) {

    Object o = new Object();
    theLogger.info("I created this object: " + o);

  }
}
```

There are various methods used to log information at different levels of severity, which are shown in Table 2-2.

Table 2-2. SLF4J logging methods

Name	Meaning
`trace`	Verbose debugging (disabled by default)
`debug`	Verbose debugging
`info`	Low-level informational message
`warn`	Possible error
`error`	Serious error

One of the advantages of SLF4J over most of the other logging APIs is the avoidance of the dead string antipattern. When using many other logger APIs, you may find code like the following:

```
logger.log("The value is " + object + "; this is not good");
```

This can lead to a performance problem, in that the object's `toString()` is implicitly called, and two string concatenations are performed, before we even know if the logger is going to use them! In code that is called repeatedly, this can lead to a lot of overhead.

This led the other logging packages to offer code guards, based on logger methods that can find out very quickly if a logger is enabled, leading to code like the following:

```
if (logger.isEnabled()) {
      logger.log("The value is " + object + "; this is not good");
}
```

This solves the performance problem but clutters the code! SLF4J's solution is to use a mechanism similar to (but not quite compatible with) Java's `MessageFormat` mechanism, as shown in Example 2-5.

Example 2-5. main/src/main/java/logging/Slf4jDemo2.java

```
public class Slf4jDemo2 {

 final static Logger theLogger = LoggerFactory.getLogger(Slf4jDemo2.class);

 public static void main(String[] args) {

  try {
   Person p = new Person();
   // populate person's fields here...
   theLogger.info("I created an object {}", p);

   if (p != null) {  // bogus, just to show logging
    throw new IllegalArgumentException("Just testing");
   }
  } catch (Exception ex) {
   theLogger.error("Caught Exception: " + ex, ex);
  }
 }
}
```

Although this doesn't actually demonstrate network logging, it is easy to accomplish that in conjunction with a logging implementation like Log4j or JUL (a standard part of the JDK), which allows you to provide configurable logging. Log4j is described in the next recipe.

See Also

The SLF4J website contains a manual that discusses the various `CLASSPATH` options. There are also some Maven artifacts for the various options.

2.13 Network Logging with Log4j

Problem

You wish to write logfile messages using Log4j.

Solution

Get a `Logger` and use its `log()` method or the convenience methods. Control logging by changing a properties file. Use the org.apache.logging.log4j.net package to make it network based.

Discussion

Logging using Log4j is simple, convenient, and flexible. You need to get a `Logger` object from the static method `LogManager.getLogger()`. The `Logger` has public void methods (`debug()`, `info()`, `warn()`, `error()`, and `fatal()`), each of which takes one `Object` to be logged (and an optional `Throwable`). As with `System.out.println()`, if you pass in anything that is not a `String`, its `toString()` method is called. A generic logging method is also included:

```
public void log(Level level, Object message);
```

The `Level` class is defined in the Log4j 2 API. The standard levels are, in order, `DEBUG` < `INFO` < `WARN` < `ERROR` < `FATAL`. That is, debug messages are considered the least important, and fatal messages the most important. Each `Logger` has a level associated with it; messages whose level is less than the `Logger`'s level are silently discarded.

WARNING

This recipe describes Version 2 of the Log4j API. Between Version 1 and Version 2, there were changes to the package names, filenames, and the method used to obtain a `Logger`. If you see code using, for example, `Logger.getLogger("class name")` rather than the newer `LogManager.getLogger()`, that code is written to the older API, which is no longer maintained (the Log4j website refers to Log4j 1.2, and versions up to 2.12, as "legacy"; we are using 2.13 in this recipe). However, a good degree of compatibility is offered for code written to the 1.x API; see the Log4J documentation.

A simple application can log messages using these few statements:

```
public class Log4JDemo {

  private static Logger myLogger = LogManager.getLogger();

  public static void main(String[] args) {

    Object o = new Object();
    myLogger.info("I created an object: " + o);

  }
}
```

If you compile and run this program with no *log4j2.properties* file, it does not produce any logging output (see the *log4j2demos* script in the source folder). We need to create a configuration file whose default name is *log4j2.properties*. You can also provide the logfile name via system properties: `-Dlog4j.configurationFile=URL`.

TIP

Log4j configuration is very flexible, and therefore very complex. Even Apache's own documentation admits that "trying to configure Log4j without understanding [the logging architecture] will lead to frustration." See the Apache website for full details on the logging configuration file location and format.

Every `Logger` has a `Level` to specify what level of messages to write. It will also have an `Appender`, which is the code that writes the messages out. A `ConsoleAppender` writes to `System.out`, of course; other loggers write to files, operating system–level loggers, and so on. A simple configuration file looks something like this:

```
# WARNING - log4j2.properties must be on your CLASSPATH,
# not necessarily in your source directory.

# The configuration file for Version 2 is different from V1!

rootLogger.level = info
rootLogger.appenderRef.stdout.ref = STDOUT

appender.console.name = STDOUT
appender.console.type = Console
appender.console.layout.type = PatternLayout
```

```
appender.console.layout.pattern = %m%n
appender.console.filter.threshold.type = ThresholdFilter
appender.console.filter.threshold.level = debug
```

This file gives the root logger a level of `DEBUG`, which causes it to write all messages. The config file also sets up an appender named `STDOUT`, which is configured on the next few lines. Note that I didn't have to refer to the `com.darwinsys Logger`. Because every `Logger` inherits from the root logger, a simple application needs to configure only the root logger. The properties file can also be an XML document, or you can write your own configuration parser (almost nobody does this).

WARNING

If the logging configuration file is not found, the root logger defaults to `Level.ERROR`, so you will not see any output below the `ERROR` level.

With the configuration file in place, the demonstration works better. Running this program (with the appropriate `CLASSPATH` as done in the scripts) produces this output:

```
$ java Log4j2Demo
I created an object: java.lang.Object@477b4cdf
$
```

A common use of logging is to show a caught `Exception`, as shown in Example 2-6.

Example 2-6. main/src/main/java/Log4JDemo2.java (Log4j—catching and logging)

```
public class Log4JDemo2 {

 private static Logger myLogger = LogManager.getLogger();

 public static void main(String[] args) {

  try {
   Object o = new Object();
   myLogger.info("I created an object: " + o);
   if (o != null) {  // bogus, just to show logging
    throw new IllegalArgumentException("Just testing");
   }
  } catch (Exception ex) {
   myLogger.error("Caught Exception: " + ex, ex);
  }
 }
}
```

When run, `Log4JDemo2` produces the expected output:

```
$ java Log4JDemo2
I created an object: java.lang.Object@477b4cdf
Caught Exception: java.lang.IllegalArgumentException: Just testing
java.lang.IllegalArgumentException: Just testing
      at logging.Log4JDemo2.main(Log4JDemo2.java:17) [classes/:?]
$
```

Much of the flexibility of Log4j 2 stems from its use of external configuration files; you can enable or disable logging without recompiling the application. A properties file that eliminates most logging might have this entry:

```
rootLogger.level = fatal
```

Only fatal error messages print; all levels less than that are ignored.

To log from a client to a server on a remote machine, the `SocketAppender` can be used. There is also an `SmtpAppender` to send urgent notices via email. See the Log4j documentation for details on all the supported appenders. Here is *log4j2-network.properties*, the

socket-based networking version of the configuration file:

```
# WARNING - log4j2.properties must be on your CLASSPATH,
# not necessarily in your source directory.

# The configuration file for Version 2 is different from V1!

rootLogger.level = info
rootLogger.appenderRef.stdout.ref = STDOUT

appender.console.type = Socket
appender.console.name = STDOUT
appender.console.host = localhost
appender.console.port = 6666
appender.console.layout.type = PatternLayout
appender.console.layout.pattern = %m%n
appender.console.filter.threshold.type = ThresholdFilter
appender.console.filter.threshold.level = debug
```

This file gets passed to the demo programs via a Java system property in the `netdemos` script:

```
build=../../../../target/classes
log4j2_jar=\
${HOME}/.m2/repository/org/apache/logging/log4j/log4j-api/2.13.0/log4j-api-2.13.0.jar:\
${HOME}/.m2/repository/org/apache/logging/log4j/log4j-core/2.13.0/log4j-core-2.13.0.jar

echo "==> Log4JDemo"
java -Dlog4j.configurationFile=log4j2-network.properties \
    -classpath ".:${build}:${log4j2_jar}" logging.Log4JDemo

echo "==> Log4JDemo2"
java -Dlog4j.configurationFile=log4j2-network.properties \
    -classpath ".:${build}:${log4j2_jar}" logging.Log4JDemo2
```

When run with the *log4j2-network.properties* file, you have to arrange for a listener on the other end. On Unix systems the `nc` (or `netcat`) program will work fine:

```
$ nc -kl 6666
I created an object: java.lang.Object@37ceb1df
I created an object: java.lang.Object@37ceb1df
Caught Exception: java.lang.IllegalArgumentException: Just testing
java.lang.IllegalArgumentException: Just testing
    at logging.Log4JDemo2.main(Log4JDemo2.java:17) [classes/:?]
^C
$
```

Netcat option `-l` says to listen on the numbered port; `-k` tells it to keep listening, that is, to reopen the connection when the client closes it, as happens when each demo program exits.

There is a performance issue with some logging calls. Consider some expensive operation, like a `toString()` or two along with several string concatenations passed to a `Log.info()` call in an often-used piece of code. If this is placed into production with a higher logging level, all the work will be done, but the resultant string will never be used. In older APIs we used to use "code guards," methods like `isLoggerEnabled(Level)`, to determine whether to bother creating the string. Nowadays, the preferred method is to create the string inside a lambda expression (see Chapter 9). All the log methods have an overload that accepts a `java.util.function.Supplier` argument (Example 2-7).

Example 2-7. main/src/main/java/logging/Log4JLambda.java

```
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

public class Log4JLambda {
```

```
  private static Logger myLogger = LogManager.getLogger();

  public static void main(String[] args) {

    Person customer = getPerson();
    myLogger.info( () ->
      "Value %d from Customer %s".formatted(customer.value, customer) );

  }
```

This way the string operations will only be performed if needed: if the `Logger` is operating at the `INFO` level it will call the `Supplier` and if not, it won't do the expensive operation.

When run as part of the *log4j2demos* script, this prints:

```
Value 42 from Customer Customer[Robin]
```

For more information on Log4j, visit its main website. Log4j 2 is free software, distributed under the Apache Software Foundation license.

2.14 Network Logging with java.util.logging

Problem

You wish to write logging messages without having to add any logging dependencies.

Solution

Using the Java logging mechanism in `java.util.logging`, get a `Logger`, and use it to log your messages and/or exceptions.

Discussion

The Java Logging API (package java.util.logging) is similar to, and was obviously inspired by, the Log4j package, which it was presumably intended to replace, or at least standardize. It's a bit less flexible than Log4j, but has the advantage that it's in the JDK, so no dependencies need to be added for it.

You acquire a `Logger` object by calling the static `Logger.getLogger()` with a descriptive `String`. You then use instance methods to write to the log; these methods include the following:

```
public void log(java.util.logging.LogRecord);
public void log(java.util.logging.Level,String);
// and a variety of overloaded log( ) methods
public void logp(java.util.logging.Level,String,String,String);
public void logrb(java.util.logging.Level,String,String,String,String);

// Convenience routines for tracing program flow
public void entering(String,String);
public void entering(String,String,Object);
public void entering(String,String,Object[]);
public void exiting(String,String);
public void exiting(String,String,Object);
public void throwing(String,String,Throwable);

// Convenience routines for log( ) with a given level
public void severe(String);
public void warning(String);
public void info(String);
public void config(String);
public void fine(String);
public void finer(String);
public void finest(String);
```

As with Log4j, every `Logger` object has a given logging level, and messages below that level are silently discarded:

```
public void setLevel(java.util.logging.Level);
public java.util.logging.Level getLevel( );
public boolean isLoggable(java.util.logging.Level);
```

As with Log4j, objects handle the writing of the log. Each `Logger` has a `Handler`:

```
public synchronized void addHandler(java.util.logging.Handler);
public synchronized void removeHandler(java.util.logging.Handler);
public synchronized java.util.logging.Handler[] getHandlers( );
```

Each `Handler` has a `Formatter`, which formats a `LogRecord` for display. By providing your own `Formatter`, you have more control over how the information being passed into the log gets formatted.

Unlike Log4j, the Java SE logging mechanism has a default configuration, so Example 2-8 is a minimal logging example program.

Example 2-8. main/src/main/java/logging/JulLogDemo.java

```
import java.util.logging.Logger;

public class JulLogDemo {
 public static void main(String[] args) {

  Logger myLogger = Logger.getLogger("com.darwinsys");

  Object o = new Object();
  myLogger.info("I created an object: " + o);
 }
}
```

Running it prints the following:

```
$ juldemos
Jan 31, 2020 1:03:27 PM logging.JulLogDemo main
INFO: I created an object: java.lang.Object@5ca881b5
$
```

As with Log4j, one common use is in logging caught exceptions; the code for this is in Example 2-9. This starts by loading a configuration file, shown following the example.

Example 2-9. main/src/main/java/logging/JulLogDemo2.java (catching and logging an exception)

```
public class JulLogDemo2 {
 public static void main(String[] args) {

  // Must be a full path; does not use getResourceAsStream()
  final Path path = Path.of("target/classes" + "/" +
    "logging.properties");
  if (!Files.exists(path)) {
   System.out.printf("path %s non-existent\n", path);
  }
  System.setProperty("java.util.logging.config.file", path.toString());
  System.out.println(System.getProperty("java.util.logging.config.file"));

  Logger logger = Logger.getLogger("com.darwinsys");

  try {
   Object o = new Object();
   logger.info("I created an object: " + o);
   if (o != null) {  // just a demo to show exception logging
    throw new IllegalArgumentException("Just testing");
   }
```

```
    } catch (Exception t) {
      logger.log(Level.SEVERE, "Caught Exception", t);
    }
  }
}
```

Note that the `logging.properties` must be specified as a full path, so you have to know where it will be located at runtime. Maven and IntelliJ copy files from *src/main/resources* into the standard output folder, *target/classes*, so that is where we specify the path for the file. Here is the configuration file:

```
# Default handler
handlers=java.util.logging.ConsoleHandler,java.util.logging.SocketHandler

# Use a short log formatter (server apps often add timestamp &c).
java.util.logging.ConsoleHandler.formatter=logging.JULLogFormatter
java.util.logging.SocketHandler.formatter=logging.JULLogFormatter
java.util.logging.SocketHandler.host=127.0.0.1
java.util.logging.SocketHandler.port=5678

# Levels are: FINEST, FINER, FINE, CONFIG, INFO, WARNING, SEVERE
# ALL and NONE are potentially useful pseudo-levels.
# NONE can not be set for a logger level

# Default global logging level.
# Loggers and Handlers may override this level
.level=INFO

# Loggers are usually attached to packages.
# The global level is used by default, so levels
# specified here simply act as an override.
com.darwinsys.level=FINE
com.darwinsys.ui.level=ALL
com.darwinsys.model.level=CONFIG
com.darwinsys.data.level=SEVERE
com.darwinsys.jullambda.level=FINE
```

Because the code asks for the logger `com.darwinsys`, it will log with a level of `FINE`, analogous to `DEBUG` in `log4j`.

The logging output shows up on the standard output as:

```
Oct 14, 2024 7:24:16 PM logging.JulLogDemo2 main
INFO: I created an object: java.lang.Object@7506e922
Oct 14, 2024 7:24:16 PM logging.JulLogDemo2 main
SEVERE: Caught Exception
java.lang.IllegalArgumentException: Just testing
        at logging.JulLogDemo2.main(JulLogDemo2.java:28)
```

Because we also configured a socket handler, it will also show up on the TCP port, by default in (verbose!) XML. For this demo I used netcat (`nc`) listening on the port in the config file:

```
$ nc -l 5678
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!DOCTYPE log SYSTEM "logger.dtd">
<log>
<record>
  <date>2024-10-14T23:19:46.227009859Z</date>
  <sequence>0</sequence>
  <logger>com.darwinsys</logger>
  <level>INFO</level>
  <class>logging.JulLogDemo2</class>
  <method>main</method>
  <message>I created an object: java.lang.Object@7506e922</message>
</record>
...
```

As with Log4j, `java.util.logging` accepts a lambda expression (and has since Java 8); see Example 2-10.

Example 2-10. main/src/main/java/logging/JulLambdaDemo.java

```
/** Demonstrate how Java 8 Lambdas avoid extraneous object creation
 * @author Ian Darwin
 */
public class JulLambdaDemo {
  public static void main(String[] args) {

    Logger myLogger = Logger.getLogger("com.darwinsys.jullambda");

    Object o = new Helper();

    // If you change the log call from finest to info,
    // you see both the systrace from the toString,
    // and the logging output. As it is here,
    // you don't see either, so the toString() is not called!
    myLogger.finest(() -> "I created this object: " + o);
  }

  static class Helper {
    public String toString() {
      System.out.println("JulLambdaDemo.Helper.toString()");
      return "failure!";
    }
  }
}
```

2.15 Maintaining Your Code with Continuous Integration

Problem

You want to be sure that your entire codebase compiles and passes its tests periodically.

Solution

Use a continuous integration (CI) server such as Jenkins or TeamCity.

Discussion

If you haven't previously used CI, you are going to wonder how you got along without it. CI is simply the practice of having all developers on a project periodically *integrate* (e.g., commit) their changes into a single master copy of the project's source and then build and test the project to make sure it still works and passes its tests. This might be a few times a day, or every few days, but should not be more than that or else the integration will likely run into larger hurdles where multiple developers have modified the same file.

But it's not just big projects or program-code projects that benefit from CI. Even on a one-person project, it's great to have a single button you can click that will check out the latest version of everything, compile it, link or package it, run all the automated tests, and give a red or green pass/fail indicator. Better yet, it can do this automatically every day or even on every commit to the master branch. If you have a number of small websites, putting them all under CI control is one of several ways of moving toward an automated, DevOps culture around website deployment and management.

If you are new to the idea of CI, I can do no better than to plead with you to read Martin Fowler's insightful (as ever) paper on the topic. One of the key points is to automate both the *management* of the code *and* all the other artifacts needed to build your project, and to automate the actual process of *building* it, possibly using one of the build tools discussed earlier in this chapter.[1]

There are many CI servers, both free and commercial. In the open source world, GitHub Actions, Jenkins, and CruiseControl are some of the better known CI servers that you deploy yourself. There are also commercial/hosted solutions such as Travis CI, TeamCity, or CircleCI. TeamCity is from the developers of IntelliJ IDEA, so it probably has the best integration with the IntelliJ IDEs. These hosted servers eliminate the need for setting up and running your own CI server. They also tend to have their configuration right in your repo (*travis.yml*, etc.) so deployment to them is simplified.

Jenkins runs as a web application, either inside a Jakarta EE server or in its own standalone web server. Once it's started, you can use any standard web browser as its user interface. Installing and starting Jenkins can be as simple as unpacking a distribution and invoking it as follows:

```
java -jar jenkins.war
```

This will start up its own tiny web server. If you do that, *be sure to configure security* if your machine is reachable from the internet!

Many people find it more secure to run Jenkins in a full-function Java EE or Java web server; anything from Tomcat to JBoss to WebSphere or WebLogic will do the job and let you impose additional security constraints.

Once Jenkins is up and running and you have enabled security and are logged in on an account with sufficient privilege, you can create *jobs*. A job usually corresponds to one project, both in terms of origin (one source code checkout) and in terms of results (one *.war* file, one executable, one library, one whatever). Setting up a project is as simple as clicking the New Job button at the top left of the dashboard, as shown in Figure 2-3.

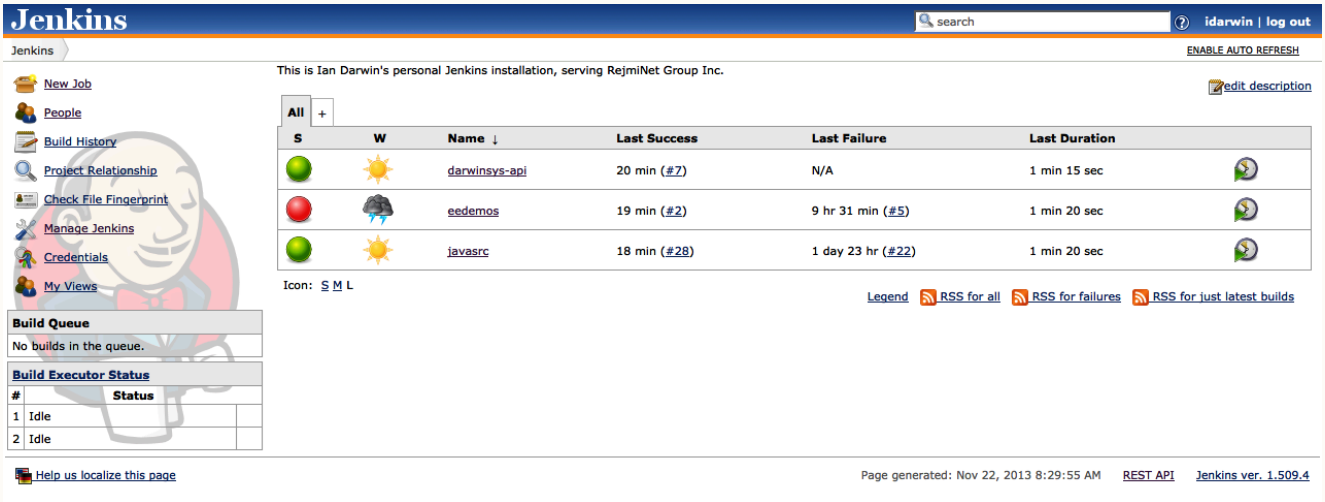


Figure 2-3. The dashboard in Jenkins

Figure 2-4 shows the first few steps of setting up a new job. You can fill in the first few pieces of information: the project's name and a brief description. Note that each and every input field has a question mark icon beside it, which will give you hints as you go along. Don't be afraid to peek at these hints!

In the next few sections of the form, Jenkins uses dynamic HTML to make entry fields appear based on what you've checked. My demo project "TooSmallToFail" starts off with no Source Code Management (SCM) repository, but your real project is probably already in Git, Subversion, or some other SCM. Don't worry if yours is not listed; there are hundreds of plug-ins to handle almost any SCM. Once you've chosen your SCM, you will enter the parameters to fetch the project's source from that SCM repository, using text fields that ask for the specifics needed for that SCM: a URL for Git, a CVSROOT for CVS, and so on.

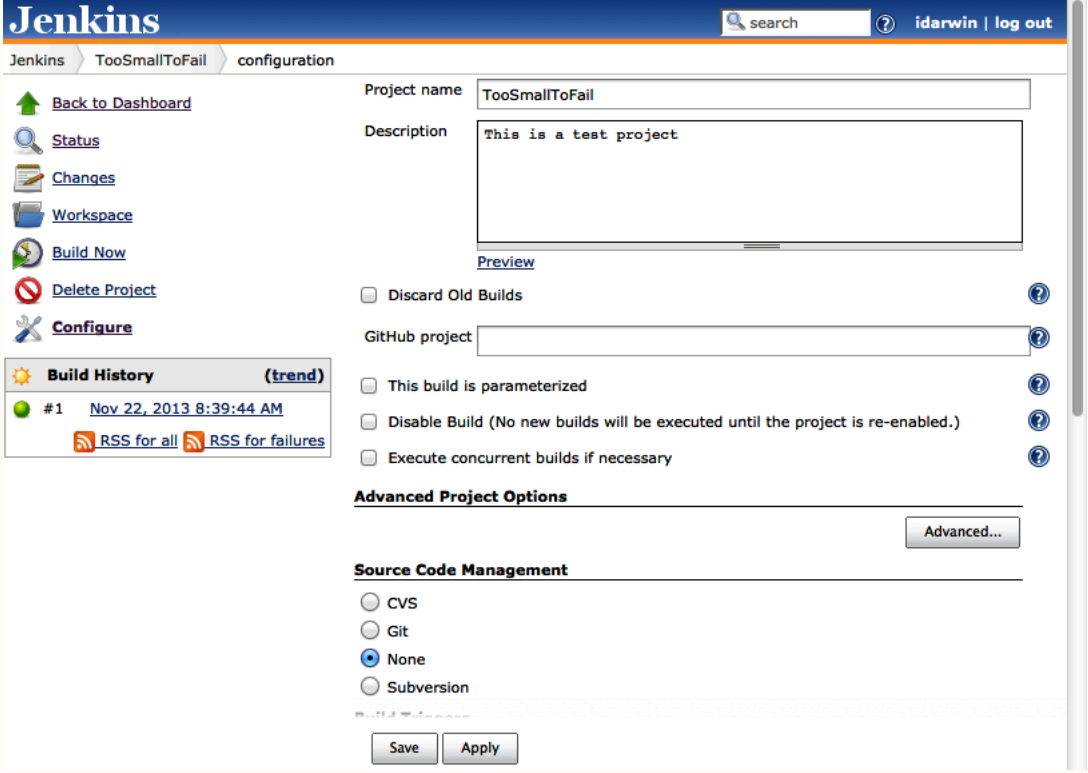


Figure 2-4. Creating a new job in Jenkins

You also have to tell Jenkins *when* and *how* to build (and package, test, deploy...) your project. For the *when*, you have several choices such as building it after another Jenkins project, building it every so often based on a cron-like schedule, or based on polling the SCM to see if anything has changed (using the same cron-like scheduler). If your project is at GitHub (not just a local Git server), or some other SCMs, you can have the project built whenever somebody pushes changes up to the repository. It's all a matter of finding the right plug-ins and following the documentation for them.

Then we have the *how*, or the build process. Again, a few build types are included with Jenkins, and many more are available as plug-ins: I've used Apache Maven, Gradle, the traditional Unix `make` tool, and even shell or command lines. As before, text fields specific to your chosen tool will appear once you select the tool. In the toy example, `TooSmallToFail`, I just use the shell command `/bin/false` (which should be present on any Unix or Linux system) to ensure that the project does, in fact, fail to build, just so you can see what that looks like.

You can have zero or more build steps; just keep clicking the Add button and add additional ones, as shown in Figure 2-5.

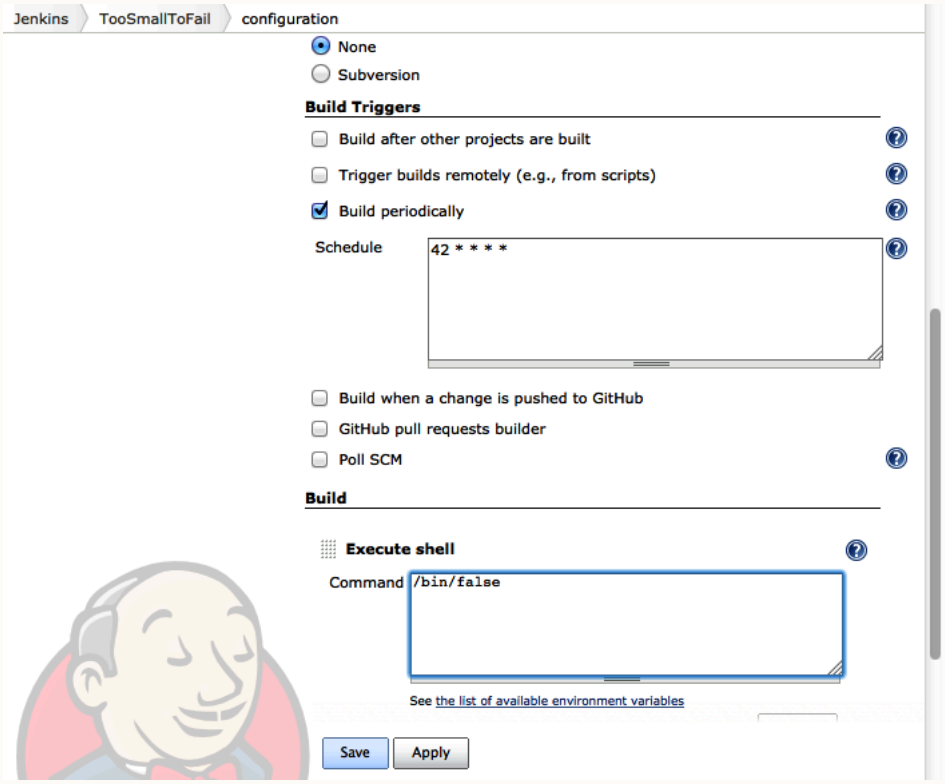


Figure 2-5. Configuration for SCM and adding build steps in Jenkins

Once you think you've entered all the necessary information, click the Save button at the bottom of the page, and you'll go back to the project's main page. Here you can click the funny little Build Now icon at the far left to initiate a build right away. Or if you have set up build triggers, you could wait until they kick in; but then again, wouldn't you rather know right away whether you've got it just right? Figure 2-6 shows the build starting.

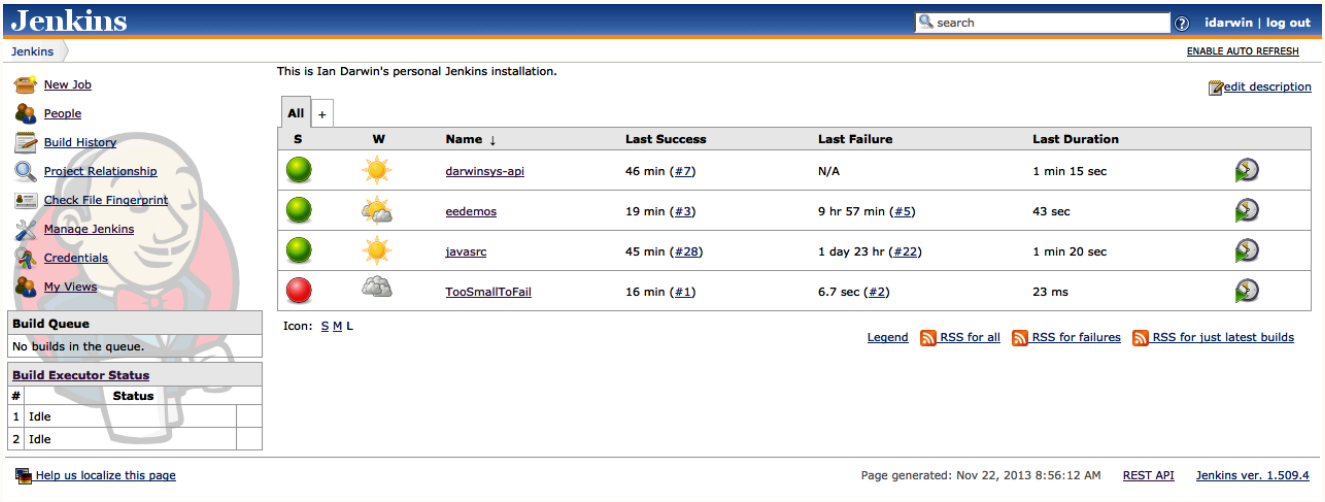


Figure 2-6. After a new job is added in Jenkins

Should a job fail to build, you get a red ball instead of a green one. Actually, a successful build shows a blue ball by default (the *go* bulb in many traffic lights in Japan, where Hudson/Jenkins's original author Kohsuke Kawaguch lives, is blue rather than green), but most people outside Japan prefer green for success, so the optional Green Balls plug-in is often one of the first to be added to a new installation.

Beside the red or green ball, you will see a "weather report" ranging from sunny (the last several builds have succeeded) to cloudy, rainy, or stormy (no recent builds have succeeded).

Click the link to the project that failed, and then the link to Console Output, and figure out what went wrong. The usual workflow is then to make changes to the project, commit/push them to the source code repository, and that may trigger the Jenkins build again.

There are *hundreds* of optional plug-ins for Jenkins. To make your life easier, almost all of them can be installed by clicking the Manage Jenkins link and then going to Manage Plug-ins. The Available tab lists all the plug-ins that are available from *Jenkins.io*; you just need to click the checkbox beside the ones you want, and click Apply. You can also find updates there. If your plug-in addition or upgrade requires a restart, you'll see a yellow ball and words to that effect; otherwise you should see a green (or blue) ball indicating plug-in success. You can also see the list of plug-ins directly on the web.

If you use a different CI system, you'll need to check that system's documentation, but the concepts and the benefits will be similar.

2.16 Performance Timing

Problem

Slow performance?

Solution

Use a *profiler*, or time individual methods using `System.currentTimeMillis()` before and after invoking the target method; the difference is the time that method took.

Discussion

Profilers

Profiling tools—profilers—have a long history as one of the important tools in a programmer's toolkit. A commercial profiling tool will help find bottlenecks in your program by showing both the number of times each method was called and the amount of time in each.

Quite a bit of useful information can be obtained from a Java application by use of the `VisualVM` tool, which was part of the Oracle JDK up until Java 8. With Java 9 this tool was released as open source, and it's now available from the VisualVM project.

Another tool that is part of the JDK is Java Flight Recorder, which is now open source and built into the JDK. Its data is meant to be analyzed by Java Mission Control. There are also third-party profilers that will give more detailed information; a web search will find current commercial offerings.

Measuring a single method

The simplest technique is to save the JVM's accumulated time before and after dynamically loading a main program and then calculate the difference between those times. Code to do just this, using the Reflection API, is presented in Example 17-26; for now, just remember that we have a way of timing a given Java class.

One way of measuring the efficiency of a particular operation is to run it many times in isolation. The overall time the program takes to run thus approximates the total time of many invocations of the same operation. Gross numbers like this can be compared if you want to know which of two ways of doing something is more efficient. Consider the case of string concatenation versus `println()`. The code:

```
println("Time is " + n.toString( ) + " seconds");
```

will probably work by creating a `StringBuilder`; appending the string `"Time is"`, the value of `n` as a string, and `"seconds"`; and finally converting the finished `StringBuilder` to a `String` and passing that to `println()`. Suppose you have a program that does a lot of this, such as a Java servlet that creates a lot of HTML this way, and you expect (or at least hope) your website to be sufficiently busy so that doing this efficiently will make a difference. There are two ways of thinking about this:

- Theory A: this string concatenation is inefficient.
- Theory B: string concatenation doesn't matter; `println()` is inefficient, too.

A proponent of Theory A might say that because `println()` just puts stuff into a buffer, it is very fast and that string concatenation is the expensive part.

How to decide between Theory A and Theory B? Assume you are willing to write a simple test program that tests both theories. Let's just write a simple program both ways and time it. Example 2-11 is the timing program for Theory A.

Example 2-11. main/src/main/java/performance/StringPrintA.java

```
public class StringPrintA {
  public static void main(String[] argv) {
    Object o = "Hello World";
    for (int i=0; i<100000; i++) {
      System.out.println("<p><b>" + o.toString() + "</b></p>");
    }
  }
}
```

`StringPrintAA` (in the *javasrc* repo but not printed here) is the same but explicitly uses a `StringBuilder` for the string concatenation. Example 2-12 is the tester for Theory B.

Example 2-12. main/src/main/java/performance/StringPrintB.java

```
public class StringPrintB {
  public static void main(String[] argv) {
    Object o = "Hello World";
    for (int i=0; i<100000; i++) {
      System.out.print("<p><b>");
      System.out.print(o.toString());
      System.out.print("</b></p>");
      System.out.println();
    }
  }
}
```

Timing results

I ran `StringPrintA`, `StringPrintAA`, and `StringPrintB` twice each on the same computer. To eliminate JVM startup times, I ran them from a program called `TimeNoArgs`, which takes a class name and invokes its `main()` method, using the Reflection API. `TimeNoArgs` and a shell script to run it, *stringprinttimer.sh*, are in the *performance* folder of the *javasrc* source repository. Here are the results:

2004 program	Seconds
`StringPrintA`	17.23, 17.20 seconds
`StringPrintAA`	17.23, 17.23 seconds
`StringPrintB`	27.59, 27.60 seconds

2014 program	Seconds
`StringPrintA`	0.714, 0.525 seconds
`StringPrintAA`	0.616, 0.561 seconds
`StringPrintB`	1.091, 1.039 seconds

2024 program	Seconds
`StringPrintA`	0.146, 0.075 seconds
`StringPrintAA`	0.098, 0.08 seconds
`StringPrintB`	0.298, 0.282 seconds

Although the times went down by around two orders of magnitude over two decades due to JVM improvements and faster hardware (Moore's Law), the ratios remain consistent: `StringPrintB`, which calls `print()` and `println()` multiple times, takes very roughly twice as long.

Moral: don't guess. If it matters, time it.

Another moral: multiple calls to `System.out.print()` cost more than the same number of calls to a `StringBuilder`'s `append()` method, by a factor of roughly 1.5 (or 150%). Theory B wins; the extra `println` calls appear to save a string concatenation but make the program take substantially longer.

Other aspects of performance: Garbage collection

There are many other aspects of software performance. One that is fundamental to Java is garbage collection (GC) behavior. Sun/Oracle usually discusses this at JavaOne. For example, see the 2003 JavaOne presentation "Garbage Collection in the Java HotSpot Virtual Machine". See also the 2007 JavaOne talk by the same GC development team, "Garbage Collection-Friendly Programming," TS-2906. JavaOne 2010 featured an updated presentation entitled "The Garbage Collection MythBusters".

2.17 Creating a Custom JDK Distribution with jlink

Problem

You are distributing your application to end users, and you want to minimize the size of your download.

Solution

Modularize your application (see Recipe 2.3), use `jdeps` to get a complete list of the modules it uses, then use `jlink` to create the mini-Java, and distribute that to your users.

Discussion

`jlink` is a command-line tool introduced in Java 9 that can make up a mini-Java distribution containing only your application and the JDK classes it uses. That is, it omits any of the thousands of JDK classes that your app will never use.

First, you need to compile and package your `module-info` and your application code. You can use Maven or Gradle, or just use the JDK tools directly:

```
$ javac -d . src/*.java
$ jar cvf demo.jar module-info.class  demo
```

If you wish to see the list of modules that will be included, you can optionally run the `jdeps` tool to get this list:

```
$ jdeps --module-path . demo.jar
demo
 [file:///Users/ian/workspace/javasrc/jlink/./]
   requires mandated java.base (@11.0.2)
demo -> java.base
   demo         -> java.io       java.base
   demo         -> java.lang     java.base
```

Once the classes have been compiled, you can run the `jlink` tool to build a mini-Java distribution with your demo app embedded:

```
jlink --module-path . --no-header-files \
    --no-man-pages --compress=2 --strip-debug \
    --launcher rundemo=demo/demo.Hello \
    --add-modules demo --output mini-java
```

The `--launcher` *name=module/main* argument asks `jlink` to create a script file named *name* to run the application.

Not all options will be needed. In one application of mine, I did the following, which also shows the disk savings by not including the entire JDK:

```
$ jlink --output myapp-runtime --add-modules java.base,java.desktop,java.logging
$ du -sh myapp-runtime/
252M    myapp-runtime/
$ du -sh /usr/local/jdk-21
381M    /usr/local/jdk-21
$
```

There was a 33% reduction in the size of the regular JDK after creating the mini-JDK.

If you got no errors, you should be able to run the mini-JDK either with the `java` command or with the generated shell script:

```
$ mini-java/bin/java demo.Hello
Hello, world.
$ mini-java/bin/rundemo
Hello, world.
$
```

You might want to copy the entire mini-Java folder to a machine that doesn't have a regular Java installation and run it there in order to be sure you don't have any missing dependencies.

WARNING

The concept of a mini-distribution is appealing, but you must consider these issues:

- Any non-modularized libraries will cause failure.
- There is no upgrade mechanism for such mini-Javas. These are quite suitable for microservice deployments where you rebuild often. For applications shipped to customers, though, you'd have to regenerate them and get your customers to download and reinstall (on short notice whenever there's a security update).
- Disk space is generally no longer expensive relative to the cost of your time in maintaining such a distribution.

Thus, you have to decide if this is worthwhile for your application.

2.18 Creating Platform-Specific Installers with jpackage

Problem

14 You have a desktop or command-line application that users will download and install on their systems. You want your users to be able to install your app using platform-appropriate installers such as `rpm` / `deb` on Linux, `dmg` on macOS, and `MSI` on Microsoft Windows, and without their having to download specific versions of Java.

Solution

Use `jpackage`, which creates platform-specific installers that include a customized JDK that will include the APIs needed to run your application.

Discussion

Java 14 introduced two tools, `jlink` and `jpackage` (they were in preview before this). The `jpackage` command can create *.msi* or *.exe* installers for Microsoft Windows, *pkg* or *dmg* files for macOS, and *app-image*, *rpm*, or *deb* files for Linux. These installers include your Java *.class* files, JAR file dependencies, and the parts of the JDK needed to run your app. This saves users from having to download a JDK, keep track of different versions of Java for different apps, etc. It trades off a bit of disk space to get rid of a lot of maintenance and incompatibility issues.

Unsurprisingly, `jpackage` must be run on the platform for which you want to make an installer, because it's going to copy parts of the running JDK into the installer. And because it works on several operating systems, it has quite a few command-line arguments, some general and some platform-specific. A complete list would take too much space here, but it can be found in the JDK documentation.

On Microsoft Windows you need to have installed an installer maker, which you will be prompted to install the first time you run `jpackage` on that OS.

The following is the `mkinstaller` script I use to prepare installers for `pdfshow`, a desktop app that I wrote and maintain. The complete version of the script is in this GitHub repository, and the same script gets run on all three major desktop platforms. If you want to adopt my script on Microsoft Windows, you'd need a Unix-like shell such as the `git bash` download. This will be less work than rewriting it into a Windows-only PowerShell script. I've omitted the part of the script that sets numerous *shell variables* such as the installer format (`fmt`) and icon format (`icon_format`) to use based on the operating system, and some platform-specific options (`OS_SPECIFIC`). The key part of the script is shown here:

```
jpackage \
--name PDFShow \
--app-version ${RELEASE_VERSION} \
--license-file LICENSE.txt \
--vendor "Rejminet Group Inc." \
--type "${fmt}" \
--icon src/main/resources/images/logo.${icon_format} \
--input target \
--main-jar pdfshow-${RELEASE_VERSION}-jar-with-dependencies.jar \
--runtime-image pdfshow-runtime \
${OS_SPECIFIC} || {
    echo jpackage did not complete normally!
    exit 1
}
```

The last few lines may give pause if you've not used a `*NIX` shell; the `||` means "run the commands between the `{...}` delimiters—the `echo` and the `exit` —only if the `jpackage` command fails to build an installer." Much of the complexity in the full version has to do with sorting out the OS-dependent parts and handling some options I added to make the script more flexible.

Once tamed by a script like mine, `jpackage` can easily and repeatably build installers for all three major desktop operating systems.

The `jlink` command predates `jpackage` and was used to create the runtime JDK image. Nowadays it is used directly from within `jpackage`; I no longer find it useful to call `jlink` except for use with `jpackage`.

This recipe is primarily concerned with producing application packages for desktop or traditional server-style applications. To package a REST or microservice application for deployment in a web server, just use `mvn deploy` as shown in Recipe 15.7. To package a Java REST or microservice app for Docker, you can use most of the frameworks listed in Table 15-1.

[1] If the deployment or build includes a step like "Get Smith to process file X on his desktop and copy to the server," you probably don't quite get the notion of automated testing.

Previous chapter
< 1. Getting Started: Compiling and Running Java

Next chapter
3. Strings and Things >